

**Atal Bihari Vajpayee Indian Institute of
Information Technology and Management, Gwalior**

Department of Computer Science and Engineering

M.Tech Thesis Progress Report

**Quantifying SZZ-Induced Label Noise in
Just-In-Time Software Defect Prediction
under Verification-Latency-Aware
Online Evaluation**

Report on the First Stage of the Thesis Work

Submitted by

Puneet Deshwani

Roll No. 2025MCS013

M.Tech (CSE)

Under the supervision of

Dr. Santosh Singh Rathore

Department of CSE

ABV-IIITM Gwalior

October 2026

Abstract

Just-in-time software defect prediction (JIT-SDP) aims to identify defect-introducing commits at the time they are submitted, so that review and testing effort can be directed to the changes that are most likely to be faulty. The models used for this task are trained on historical commits whose labels are almost always produced by the SZZ algorithm. SZZ starts from a bug-fixing commit and traces the modified lines back through the version history to find the commits that introduced the defect. This procedure is known to make mistakes, mainly because bug-fixing commits often contain changes that are unrelated to the fix. Since the same SZZ labels are generally used for training and for testing, the effect of these mistakes is not visible in the usual experimental setup.

The aim of this thesis is to measure how the label noise introduced by SZZ affects JIT-SDP models when they are evaluated under realistic conditions. This report presents the work carried out in the first stage of the thesis. The study uses the JIT-Defects4J dataset, which contains 27,319 commits from 21 Apache Java projects together with reference labels derived from manually validated bug-fixing lines. These reference labels are used to assess both the SZZ labels and the predictions of the models.

In the first part of the work, six variants of SZZ are executed on all projects and their labels are compared with the reference labels. It is observed that the precision of the variants lies between 18.3% and 27.2%, that the two types of labelling error differ widely between the variants (false-alarm rates of 6.7% to 26.3% and miss rates of 35.9% to 73.3%), and that the variants agree much more with each other than with the reference labels. In the second part, three prediction models are trained on seven label sets and evaluated under four evaluation settings, which results in 16,380 experimental runs. The results show that random k -fold cross-validation raises the MCC of a random forest model by 0.137 compared with a chronological split, and that scoring a model against its own training labels raises it further by 0.230. When an online learner is evaluated on a commit stream with delayed labels, the model trained on the reference labels performs better than the models trained on any of the SZZ variants, by 0.034 to 0.064 MCC. The plan for the next stage of the thesis is given at the end of the report.

Keywords: just-in-time software defect prediction, SZZ algorithm, label noise, verification latency, online learning, class imbalance

Contents

Abstract	i
List of Abbreviations	ix
1 Introduction	1
1.1 Background	1
1.2 Labelling of Commits and the SZZ Algorithm	2
1.3 Motivation	3
1.4 Problem Statement	4
2 Literature Review	6
2.1 Software Defect Prediction	6
2.2 Just-in-Time Software Defect Prediction	7
2.2.1 Change-level prediction and change metrics	7
2.2.2 Learning techniques	7
2.3 Identification of Defect-Introducing Changes	8
2.3.1 The SZZ algorithm	8
2.3.2 Variants of SZZ	8
2.3.3 Empirical evaluations of SZZ	9
2.3.4 Tangled changes	10
2.4 Label Noise in Defect Prediction	10
2.5 Evaluation of Defect Prediction Models	11
2.5.1 Validation schemes and time order	11
2.5.2 Prequential evaluation	11
2.5.3 Verification latency	12
2.5.4 Performance measures	12
2.6 Online Learning for Imbalanced Data Streams	13
2.7 Learning with Noisy Labels	13
2.8 Summary	14
3 Research Gaps, Research Questions and Objectives	15
3.1 Research Gaps	15
3.2 Research Questions	16
3.3 Objectives	17

4	Dataset	18
4.1	Requirements	18
4.2	The JIT-Defects4J Dataset	19
4.3	Subject Projects	19
4.4	Reference Labels	19
4.5	Change Metrics	21
4.6	Label Sets	22
4.7	Data Preparation	23
4.8	Label Arrival Times	23
4.9	Summary	25
5	Methodology	26
5.1	Overview of the Approach	26
5.2	Generation of SZZ Labels	27
5.3	Analysis of Label Quality	28
5.3.1	Comparison with the reference labels	28
5.3.2	Noise rates	28
5.3.3	Agreement between variants	29
5.3.4	Variation across projects and sources of false negatives	29
5.4	Prediction Models	30
5.4.1	LApredict	30
5.4.2	Random forest model based on JITLine	30
5.4.3	ORB	31
5.5	Evaluation Settings	32
5.6	Simulation of Verification Latency	33
5.7	Scoring of the Predictions	33
5.8	Performance Measures	35
5.8.1	MCC and G-mean	35
5.8.2	Measures in the prequential setting	36
5.9	Statistical Analysis	36
5.9.1	Unit of analysis	36
5.9.2	Test and effect sizes	37
5.9.3	Correction for multiple comparisons	37
5.10	Additional Analyses	38
5.11	Experimental Setup	39
5.12	Summary	40
6	Results and Discussion	41
6.1	Quality of the SZZ Labels	41
6.1.1	Overall quality	41
6.1.2	Variation across projects	43
6.1.3	Agreement between the variants	44
6.1.4	Sources of the false negatives	45

6.1.5	Discussion and answer to RQ1	46
6.2	Impact on the Prediction Models	47
6.2.1	Overall results	47
6.2.2	Effect of the evaluation setting	50
6.2.3	Effect of scoring against the training labels	51
6.2.4	Effect of the label source in the online setting	53
6.2.5	Combined view of the three effects	56
6.2.6	Label delay and model adaptation	57
6.2.7	A case in which B-SZZ labels give a better batch model	58
6.2.8	Robustness checks	59
6.2.9	Discussion and answer to RQ2	60
6.3	Summary of Findings	61
7	Conclusion and Future Work	63
7.1	Conclusions	63
7.2	Threats to Validity	64
7.2.1	Construct validity	64
7.2.2	Internal validity	65
7.2.3	External validity	65
7.2.4	Conclusion validity	65
7.3	Future Work	66
7.3.1	Mechanism of the noise effect	66
7.3.2	Reducing the effect of the noise	66
7.3.3	Further directions	67
	References	68

List of Figures

1.1	General workflow of just-in-time software defect prediction.	2
1.2	Example of how SZZ identifies defect-introducing commits and how a change that is unrelated to the fix leads to a wrong label.	3
4.1	Number of commits and percentage of defect-introducing commits in each project. The dashed line marks the percentage in the whole dataset (8.54%).	21
4.2	Distribution of the verification latency. A large part of the labels arrives after the waiting period of 90 days.	24
5.1	Overview of the thesis work. The boxes with solid borders belong to the first stage, which is reported here. The boxes with dashed borders are planned for the next stage.	27
5.2	The evaluation settings. The commits of a project are ordered by time from left to right.	32
5.3	Delivery of training labels under verification latency. In case 3 the learner first receives a wrong clean label, which is corrected when the defect is fixed.	34
6.1	(a) Precision and recall of the SZZ variants with respect to the reference labels. (b) False-alarm rate and miss rate of the variants.	43
6.2	Number of correctly flagged, wrongly flagged and missed commits for each variant.	43
6.3	Distribution of precision and recall of each variant over the 21 projects.	44
6.4	Cohen's κ between the reference labels and the labels of the six SZZ variants.	45
6.5	MCC of each project under random k -fold cross-validation and under the chronological split, for models trained on the reference labels. The thick line shows the mean.	51
6.6	Difference between the self-scored and the reference-scored MCC for each SZZ variant and both batch models.	52
6.7	ORB in the prequential setting with verification latency. (a) Mean MCC for each source of training labels with both summaries. (b) Hodges–Lehmann estimate and 95% confidence interval of the difference between the reference-trained model and each SZZ-trained model (time-averaged MCC).	54

6.8	Comparison of the two summaries of the prequential MCC. (a) Distribution of the project-level values over all label sources. (b) Terminal and time-averaged value of each project for models trained on the reference labels and on B-SZZ labels.	55
6.9	MCC under increasingly realistic conditions. Each bar is labelled with the model, the training labels and the evaluation setting. All bars except the first are scored against the reference labels.	56
6.10	Mean MCC of the four combinations of learner and label delivery. The bars show one standard error over the projects.	58
6.11	Difference between the MCC of JITLine trained on B-SZZ labels and JITLine trained on the reference labels, for each project (chronological split, scored against the reference labels).	58
6.12	Estimated difference between the reference-trained and the SZZ-trained model for different fading factors. The lines show the mean over the four warm-up settings and the shaded bands show their range.	60

List of Tables

2.1	SZZ variants considered in this work.	9
2.2	Summary of the most closely related studies.	14
4.1	Projects of the JIT-Defects4J dataset. The last two columns give the number and the percentage of defect-introducing commits according to the reference labels.	20
4.2	The fourteen change metrics used as features [3].	22
4.3	Verification latency of the commits that are linked to a bug-fixing commit.	24
4.4	Availability of arrival times and latency of the positive labels of each label source.	25
5.1	Outcomes of comparing an SZZ label with the reference label.	28
5.2	Families of statistical tests in the impact analysis.	38
5.3	Factors of the experiment in the impact analysis.	39
6.1	Number of commits flagged by each SZZ variant and comparison with the reference labels.	41
6.2	Quality of the SZZ labels with respect to the reference labels. The best value in each column is shown in bold.	42
6.3	Minimum, median and maximum of precision and recall over the 21 projects.	44
6.4	False negatives of the refined variants, divided according to whether B-SZZ also misses the commit. The last column gives the share of the 1,495 true positives of B-SZZ that the variant no longer flags.	46
6.5	Mean MCC of the batch models. “Ref.” denotes scoring against the reference labels and “Self” denotes scoring against the training labels. For models trained on the reference labels both are the same.	48
6.6	Mean MCC of ORB. For the prequential setting the time-averaged and the terminal value are given, both scored against the reference labels.	48
6.7	Mean G-mean of the three models, scored against the reference labels. For ORB in the prequential setting the time-averaged value is given.	49
6.8	Random k -fold cross-validation compared with the chronological split (MCC scored against the reference labels). HL is the Hodges–Lehmann estimate of the difference, r the rank-biserial correlation, and “Projects” the number of projects in which the k -fold value is higher. The last two columns give the Holm-corrected p -values within the family and over all tests.	50

6.9	Self-scored MCC compared with reference-scored MCC for the batch models. “Projects” is the number of projects in which the self-scored value is higher. The other columns are as in Table 6.8.	52
6.10	ORB trained on the reference labels compared with ORB trained on each SZZ variant, in the prequential setting with verification latency. HL is the estimate of the difference (reference minus variant). “Projects” is the number of projects in which the reference-trained model is better.	54
6.11	ORB with reference labels in the four combinations of learner (frozen or adaptive) and label delivery (immediate or delayed). MCC on the second half of the stream.	57
6.12	Spearman correlation between project characteristics and the difference shown in Figure 6.11.	59
6.13	Main comparisons for all projects and for the 13 projects with at least 25 defect-introducing commits in the second half of their history.	61
6.14	Summary of the main findings.	62

List of Abbreviations

AG-SZZ	Annotation Graph SZZ
AUC	Area Under the ROC Curve
B-SZZ	Basic SZZ (the original algorithm)
CI	Confidence Interval
FN, FP	False Negative, False Positive
HL	Hodges–Lehmann estimate
JIT-SDP	Just-In-Time Software Defect Prediction
L-SZZ	Largest SZZ
MA-SZZ	Meta-change Aware SZZ
MCC	Matthews Correlation Coefficient
OOB	Oversampling Online Bagging
ORB	Oversampling Rate Boosting
R-SZZ	Recent SZZ
RA-SZZ	Refactoring Aware SZZ
SMOTE	Synthetic Minority Oversampling Technique
SZZ	Śliwerski–Zimmermann–Zeller algorithm
TN, TP	True Negative, True Positive

Chapter 1

Introduction

This chapter gives the background of the work. It introduces just-in-time software defect prediction, explains how the training labels for this task are obtained, and describes the issues that motivated the present study. The problem statement is given at the end of the chapter.

1.1 Background

Software systems are changed continuously during their lifetime, and some of these changes introduce defects. A defect that is found during code review can usually be corrected quickly by the developer who wrote the code. If the same defect is found after release, it has to be reported, reproduced, assigned and fixed, and it may already have affected the users of the system. For this reason, software quality assurance activities such as code review and testing try to find defects as early as possible. These activities need time and people, and in most projects it is not possible to review and test every part of the code with the same care.

Software defect prediction was proposed to help in this situation. A defect prediction model learns from the history of a project which parts of the code are likely to be defective, so that the available effort can be spent on those parts first [1, 2]. Most of the early work predicts defects at the level of files, classes or modules, and the prediction is generally made for a release of the software. Such models are useful for planning, but they have some practical limitations. A file that is predicted as defect-prone may be large, so the developer still has to find the faulty part. The prediction is made late in the development cycle, when the changes that caused the problem are no longer fresh in the mind of the developer. It is also not always clear who should inspect a file that has been modified by many developers [3].

Just-in-time software defect prediction (JIT-SDP) addresses these limitations by making the prediction for each code change (commit) instead of each file. When a developer

submits a commit, the model estimates whether this commit is likely to introduce a defect. The prediction is available immediately, it concerns a small amount of code, and it can be assigned to the author of the commit. The idea goes back to the work of Mockus and Weiss [4] and Kim et al. [5], and the formulation that is commonly used today was given by Kamei et al. [3]. Figure 1.1 shows the general workflow. The past commits of a project are described by a set of change metrics and are labelled as clean or defect-introducing. A classifier is trained on these commits and is then applied to every new commit.

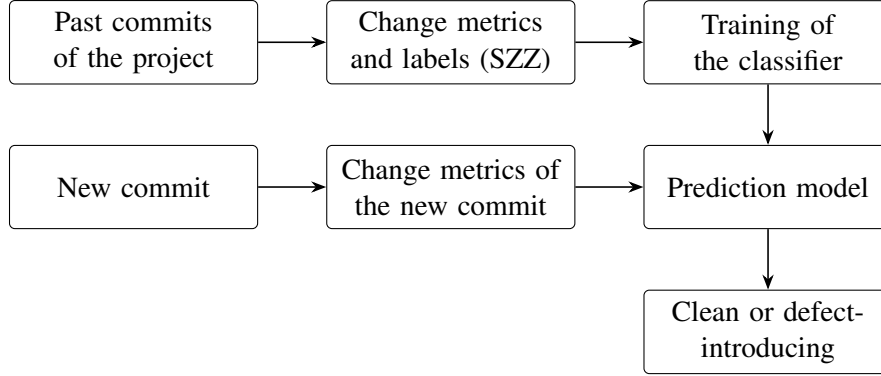


Figure 1.1: General workflow of just-in-time software defect prediction.

Three characteristics of this task are important for the present work. First, commits arrive one after another in time, so the data form a stream and the model is always used on commits that are newer than the ones it was trained on. Second, only a small fraction of the commits introduce defects. In the dataset used in this work the fraction is 8.54%, which means that the data are highly imbalanced. Third, the label of a commit is not known when the commit is made. It becomes known only later, when the defect is noticed and fixed, and for a clean commit it is never confirmed at all.

1.2 Labelling of Commits and the SZZ Algorithm

To train a JIT-SDP model, every past commit needs a label that says whether it introduced a defect. Software repositories do not store this information. What can be found in a repository and its issue tracker is the set of commits that *fixed* defects, because developers normally mention the bug report in the commit message. The commits that *introduced* the defects have to be inferred from these bug-fixing commits.

The standard method for this inference is the SZZ algorithm, proposed by Śliwerski, Zimmermann and Zeller [6]. The algorithm takes a bug-fixing commit and looks at the lines that were deleted or modified by it. For each of these lines it uses the annotation facility of the version control system (`git blame`) to find the earlier commit in which the line was last changed. These earlier commits are labelled as defect-introducing. All commits that are never reached in this way are treated as clean.

Figure 1.2 illustrates the procedure with a small example. Commit c_2 adds a faulty line. The fault is corrected later in the bug-fixing commit c_5 . When SZZ traces the corrected line back, it reaches c_2 , which is the right answer. However, the developer of c_5 has also edited another line that had nothing to do with the bug, for example to improve the formatting. This line was last changed in c_3 , so SZZ labels c_3 as defect-introducing as well, although it is clean.

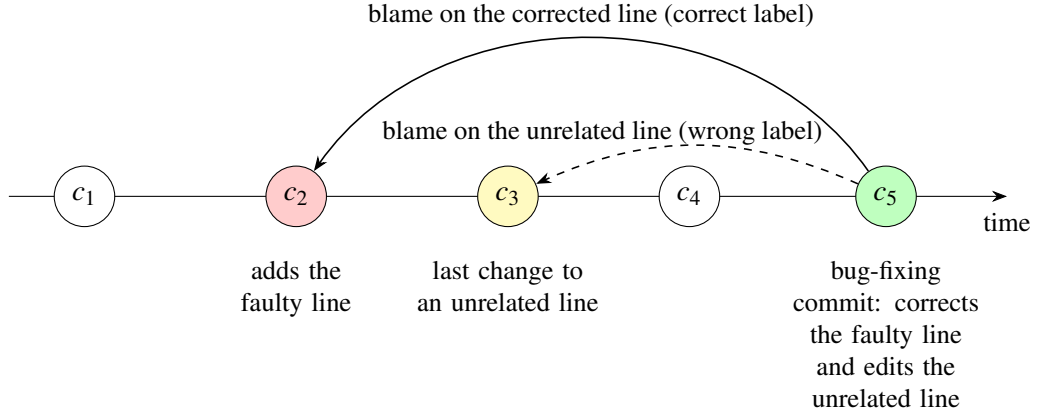


Figure 1.2: Example of how SZZ identifies defect-introducing commits and how a change that is unrelated to the fix leads to a wrong label.

A bug-fixing commit that contains such unrelated modifications is called a *tangled* commit. Tangled commits are common. Herbold et al. [7] manually examined a large number of bug-fixing commits and found that only 17% to 32% of the changed lines in them actually contribute to the bug fix. The remaining lines are changes to tests, documentation, formatting, refactoring and other modifications. Several improved versions of SZZ have been proposed to remove such lines before the blame step, and they are reviewed in Chapter 2. It is important to note that all of them still *estimate* the label. None of the labels used in JIT-SDP studies is directly observed.

1.3 Motivation

The motivation for this work comes from four issues in the way JIT-SDP models are currently built and evaluated.

Noise in the SZZ labels. Since the labels are produced by a heuristic, some clean commits are labelled as defect-introducing (false positives) and some defect-introducing commits are labelled as clean (false negatives). A model trained on such labels learns partly from wrong examples. Earlier studies have evaluated the accuracy of SZZ on small manually checked samples [8, 9], but the amount and the type of noise over the complete history of a project, which is what a JIT-SDP model is trained on, has received less attention.

Evaluation against the same noisy labels. In nearly all JIT-SDP studies the test set is labelled by the same SZZ implementation as the training set. A model is therefore rewarded when it predicts what SZZ would say, including the cases where SZZ is wrong. The reported performance then measures how well the model reproduces the labelling heuristic, which is not the same as how well it finds real defects. The difference between the two can only be measured if labels from an independent source are available for testing.

Evaluation that ignores time. Many studies evaluate their models with random k -fold cross-validation. On commit data this means that the model may be trained on commits that were made after the commits on which it is tested. Such a setting cannot occur in practice, and it has been shown for defect prediction in general that it gives optimistic results [10, 11].

Delay in the arrival of labels. Even when the commits are split by time, it is normally assumed that the label of every training commit is known at training time. In reality a defect-introducing commit is recognised only when its defect is fixed, which may happen months or years later. Cabral et al. [12] called this delay *verification latency*. During the delay the commit is treated as clean, so a model that is updated continuously receives a wrong label first and the correct label later. In the dataset used here the median delay is 113 days, and more than half of the defect labels arrive later than 90 days after the commit.

These four issues are related. The first one is about the quality of the training data, and the other three are about the realism of the evaluation. A study that wants to know how much the label noise matters in practice has to control all of them at the same time.

1.4 Problem Statement

JIT-SDP models are trained and evaluated on labels generated by the SZZ algorithm, and the evaluation is often carried out without respecting the order of commits and the delay with which labels become available. As a result, it is not known how much the errors made by SZZ reduce the ability of a model to identify real defect-introducing commits when the model is used in a realistic setting, and it is not possible to say how much of the performance reported in the literature is due to the model and how much is due to the way it was evaluated.

The problem addressed in this thesis is therefore to quantify the label noise introduced by SZZ and its variants, to measure the effect of this noise on JIT-SDP models under an evaluation that respects time order and verification latency, and, in the later part of the

thesis, to understand how the noise affects an online learner and how its effect can be reduced.

The thesis work has been planned in stages. This report presents the first stage, in which the quality of the labels produced by six SZZ variants is analysed and the effect of these labels on JIT-SDP models is measured. The questions of how the noise affects an online learner and how this effect can be reduced belong to the next stage. Only the plan for that stage is given, in Chapter 7, and no results are reported for it here.

Chapter 2

Literature Review

This chapter reviews the literature related to the present work. It starts with defect prediction in general and JIT-SDP in particular, then discusses the SZZ algorithm and its variants, the studies on label noise, the evaluation of prediction models, online learning for imbalanced data streams and learning with noisy labels. A summary of the most closely related studies is given at the end of the chapter.

2.1 Software Defect Prediction

Software defect prediction has been studied for more than three decades. The usual approach is to describe each software module by a set of metrics, such as size, complexity, coupling or the history of past changes, and to train a classifier that separates defective modules from clean ones. Hall et al. [1] carried out a systematic review of 208 fault prediction studies and observed that models based on simple techniques such as Naïve Bayes and logistic regression often perform as well as more complex ones, and that the quality of the data and of the experimental procedure has a large influence on the reported results. Rathore and Kumar [2] reviewed the field with respect to three aspects: the software metrics used, the prediction techniques and the data quality issues. Among the data quality issues they discuss class imbalance and noise in the data, both of which are central to the present work.

Most of this research predicts defects for files, classes or modules of a given release. The limitations of this granularity, mentioned in Chapter 1, led to the prediction of defects at the level of individual changes.

2.2 Just-in-Time Software Defect Prediction

2.2.1 Change-level prediction and change metrics

Mockus and Weiss [4] were among the first to predict the risk of a software change. They used properties of the change, such as its size, the number of files and subsystems it touches, the experience of the developer and the type of the change, to predict the probability that a change to a large telecommunication system would cause a failure. Kim et al. [5] introduced *change classification*, in which a classifier is trained to decide whether a change is buggy or clean. They used features extracted from the change log, the source code and the change metadata, and obtained on average 78% accuracy and 60% recall for buggy changes on twelve open-source projects.

Kamei et al. [3] carried out a large empirical study on six open-source and five commercial projects and introduced the term “just-in-time” for this kind of quality assurance. They proposed fourteen change metrics that are grouped into five dimensions: diffusion, size, purpose, history and experience. Their logistic regression model predicted defect-inducing changes with an average accuracy of 68% and a recall of 64%. These fourteen metrics have become the standard feature set for JIT-SDP and are also used in the present work. They are described in detail in Section 4.5. A recent survey by Zhao et al. [13] covers 67 JIT-SDP studies and shows that the majority of them follow the formulation of Kamei et al. with respect to both the features and the way the labels are obtained.

McIntosh and Kamei [14] studied how JIT-SDP models behave over time on the Qt and OpenStack projects. They found that the properties of defect-inducing changes are not stable. The importance of the different metrics changes as a project evolves, and a model loses a considerable part of its predictive power when it is used long after it was trained. This result shows that time has to be taken into account both in training and in evaluating such models.

2.2.2 Learning techniques

A wide range of learning techniques has been applied to JIT-SDP. Besides logistic regression, random forests and other ensemble methods are frequently used. More recently, deep learning models have been proposed that learn features directly from the content of a change. DeepJIT [15] applies convolutional networks to the commit message and to the changed code. CC2Vec [16] learns a vector representation of a code change with a hierarchical attention network and combines it with DeepJIT.

Later studies have questioned whether this additional complexity is necessary. Zeng et al. [17] replicated DeepJIT and CC2Vec on a larger dataset of more than 310,000 changes. They reported that the deep models did not consistently perform better than traditional ap-

proaches, and that a logistic regression model using only the number of added lines, which they called LApredict, outperformed both of them in most cases while being many times faster. Pornprasit and Tantithamthavorn [18] proposed JITLine, which uses a random forest together with oversampling of the minority class, and showed that it is more accurate and much faster than the deep models. Yang et al. [19] had earlier shown for effort-aware prediction that simple unsupervised models built from a single change metric can be better than supervised models.

These studies are relevant here for two reasons. First, they show that the ranking of models in JIT-SDP depends strongly on how the evaluation is done. Second, they provide two well-known models of very different complexity, LApredict and JITLine, which are used in the present work to examine whether the effect of label noise depends on the capacity of the model.

2.3 Identification of Defect-Introducing Changes

2.3.1 The SZZ algorithm

The SZZ algorithm [6] consists of two phases. In the first phase, bug-fixing commits are identified by linking the commits of the version control system to the reports in the bug tracking system, mainly by searching the commit messages for bug identifiers. In the second phase, for every bug-fixing commit the lines that were deleted or modified are determined, and the `annotate` (`blame`) command is used to find the most recent earlier commits that changed these lines. Commits that were made after the bug was reported are excluded, since they cannot have caused the bug. The remaining commits are considered to be bug-introducing.

The algorithm depends on two assumptions. The first is that the lines changed by the fix are the lines that contained the defect. The second is that the commit that last changed such a line is the commit that made it defective. Both assumptions can fail. The first one fails for tangled commits, where some of the changed lines are not related to the fix. The second one fails when a line was last touched by a cosmetic change, such as a renaming or re-indentation, so that the `blame` points to this change instead of the original faulty one. In addition, a defect that is fixed only by adding new lines leaves no deleted or modified line to trace, and a defect caused by a change in the environment or in the requirements has no introducing commit in the repository at all [20].

2.3.2 Variants of SZZ

Several variants have been proposed to reduce the number of wrong labels produced by the original algorithm, which is usually called B-SZZ (basic SZZ). Table 2.1 lists the

variants that are studied in this work.

Table 2.1: SZZ variants considered in this work.

Variant	Main idea
B-SZZ [6]	Original algorithm. Every line deleted or modified by the bug-fixing commit is traced back with blame.
AG-SZZ [21]	Uses an annotation graph to follow lines across revisions and ignores blank lines, comments and formatting changes.
MA-SZZ [8]	Builds on AG-SZZ and additionally ignores meta-changes, i.e. branch changes, merge changes and property changes, which do not modify the source code.
R-SZZ [22]	Builds on MA-SZZ and keeps, for each bug-fixing commit, only the most recent of the candidate commits.
L-SZZ [22]	Builds on MA-SZZ and keeps only the candidate commit that changed the largest number of lines.
RA-SZZ [23]	Builds on MA-SZZ and ignores lines that were changed by refactoring operations, which are detected with a refactoring detection tool. It is available for Java only.

Kim et al. [21] observed that many lines changed in a bug-fixing commit are comments, blank lines or formatting changes and proposed to ignore them. They also replaced the simple annotate command by an annotation graph, which follows a line more reliably when code is moved. Da Costa et al. [8] noticed that some commits flagged by SZZ are only changes to the repository structure, such as merges, and excluded these meta-changes. R-SZZ and L-SZZ go one step further and select a single commit from the candidates returned for a bug-fixing commit [22]. They assume that a bug has one introducing commit and choose either the latest or the largest candidate. Neto et al. [23] showed that refactoring operations are a frequent cause of wrong labels and proposed RA-SZZ, which removes refactored lines from the analysis.

All these variants try to reduce false positives by removing lines or candidate commits. An obvious consequence, which is examined in Chapter 6, is that some correct candidates may be removed as well.

2.3.3 Empirical evaluations of SZZ

The accuracy of SZZ is difficult to evaluate, because the true defect-introducing commits are not known. Da Costa et al. [8] proposed a framework that evaluates SZZ implementations indirectly through three criteria: the earliest appearance of the bug, the future impact of the flagged changes and the realism of the bug introduction. They applied it to five implementations on ten open-source projects and found that the existing implementations still have considerable room for improvement.

Rodríguez-Pérez et al. [24] reviewed 187 papers that use SZZ and found that most

of them neither discuss the limitations of the algorithm nor provide enough detail to reproduce the labelling. In a later study [20] they manually analysed how 116 bugs were introduced in two open-source projects. They found that a part of the bugs are *extrinsic*, i.e. caused by changes outside the project code, and that the SZZ-based algorithms reached F-scores of only 0.44 to 0.77 on the remaining ones.

Rosa et al. [9, 25] built a *developer-informed oracle* from commit messages in which the developer explicitly names the commit that introduced the bug being fixed. The oracle contains 1,930 such cases, extended to 2,304 in the journal version, and was used to compare several SZZ variants. R-SZZ obtained the best results, with a precision of about 66% and an F-measure of about 61%, while B-SZZ had a higher recall but a precision of only about 39%. The authors also released PySZZ, an open-source implementation of the variants, which is used in the present work.

Two properties of this oracle are important for the present study. It contains only bug-fixing commits together with their introducing commits and no clean commits, so it cannot be used to train or test a JIT-SDP model. It also evaluates SZZ only on fixes for which an introducing commit is known to exist. This is a different question from the one that matters for JIT-SDP, where every commit in the history of the project receives a label.

2.3.4 Tangled changes

Herzig and Zeller [26] studied tangled changes in five open-source Java projects and found that up to 15% of the bug fixes consist of several unrelated changes. As a result, at least 16.6% of the source files were incorrectly associated with bug reports. Herbold et al. [7] conducted a large crowd-sourced study in which every changed line of the bug-fixing commits of several Apache projects was labelled by four participants. They found that only 17% to 32% of all changed lines contribute to the bug fix. When only changes to production code files are considered, the share is 66% to 87%. For about 11% of the lines the participants could not agree on a label. The dataset produced by this study, called LLTC4J, is the basis of the reference labels used in the present work (Section 4.4).

2.4 Label Noise in Defect Prediction

The effect of wrong labels on defect prediction models has been studied mainly for file-level prediction. Kim et al. [27] added artificial false positive and false negative noise to defect datasets and found that the prediction performance decreases significantly once the datasets contain about 20% to 35% of both types of noise. They also proposed a method to detect and remove noisy instances. Tantithamthavorn et al. [28] examined mislabelling that arises from wrongly classified issue reports. They showed that such mislabelling is

not random, and that models trained on the noisy data keep their precision but reach only 56% to 68% of the recall of models trained on clean data.

For JIT-SDP, the most closely related study is that of Fan et al. [29]. They labelled 126,526 changes from ten Apache projects with four SZZ variants (B-SZZ, AG-SZZ, MA-SZZ and RA-SZZ) and examined how the differences between the variants affect JIT-SDP models. They reported that the changes mislabelled by B-SZZ and MA-SZZ do not reduce the prediction performance considerably, whereas those mislabelled by AG-SZZ lead to a statistically significant reduction. Since no ground truth was available, they treated the labels of RA-SZZ as correct and measured the mislabelling of the other variants relative to it. This has two consequences. The noise of RA-SZZ itself could not be measured, and errors that are common to all variants, for example those caused by tangled changes, remained invisible. The study also did not consider the delayed arrival of labels.

2.5 Evaluation of Defect Prediction Models

2.5.1 Validation schemes and time order

Cross-validation with random folds is the most common validation scheme in defect prediction. For data that are ordered in time it has a known weakness, because a model can be trained on data that are newer than the test data. Tan et al. [11] pointed out for change classification that cross-validation uses information from the future and can therefore report a precision that is too high. They proposed a time-sensitive evaluation in which the model is trained only on changes that precede the test changes. Falessi et al. [10] analysed this issue systematically for within-project defect classifiers. They showed that validation techniques which do not preserve the order of the data produce results that differ from those of order-preserving techniques, and recommended the latter. In spite of these findings, random cross-validation is still widely used in JIT-SDP studies [13].

2.5.2 Prequential evaluation

When data arrive as a stream, the standard evaluation procedure is *prequential* (predictive sequential) evaluation, also known as test-then-train. Each example is first used to test the model and afterwards to update it. The performance is accumulated over the stream. Since the performance at the beginning of the stream is often poor and less relevant, a forgetting mechanism such as a sliding window or a fading factor is applied. Gama et al. [30] showed that the prequential error computed with a fading factor converges to the same value as the error on an independent holdout set. This result holds for the error rate. For measures that are computed from the complete confusion matrix, such as MCC, no

corresponding result is available, which has to be kept in mind when prequential values of such measures are summarised (Section 5.8).

2.5.3 Verification latency

Cabral et al. [12] were the first to investigate JIT-SDP as an online learning problem with verification latency. In their setting a commit is used as a clean training example when a waiting period has passed without a defect being linked to it, and as a defect-inducing example as soon as a defect is linked to it. They used a waiting period of 90 days. They showed that, because of this mechanism, the proportion of the two classes seen by the learner changes over time, which they called class imbalance evolution, and that this hinders the existing approaches. To deal with it they proposed the ORB method, which is described in Section 2.6. In a later study, Cabral and Minku [31] examined concept drift in JIT-SDP in more detail. They reported that the time needed to discover a defect ranges from one day to more than eleven years, and proposed a method that also uses the predictions on unlabelled commits to detect when the model has become unreliable. Tabassum et al. [32] studied the use of data from other projects in the same online setting.

In these studies the wrong labels that the learner receives are caused by time: a defect-introducing commit is treated as clean until its defect is found. The labels that finally arrive are assumed to be correct. The noise that comes from the SZZ algorithm itself, where the final label is wrong and is never corrected, is not addressed.

2.5.4 Performance measures

Because the classes are imbalanced, accuracy is not a suitable measure for defect prediction. Precision, recall and the F-measure are widely used, but the F-measure ignores the true negatives and depends on the proportion of the positive class. Yao and Shepherd [33] showed that the use of the F-measure can lead to different conclusions than the Matthews correlation coefficient (MCC) [34] in a considerable number of defect prediction comparisons and recommended MCC. Chicco and Jurman [35] reached the same recommendation for binary classification in general. MCC uses all four cells of the confusion matrix and takes the value zero for a classifier that predicts at random or always predicts the same class. For measuring the agreement between two sets of labels, Cohen's κ [36] is commonly used, which corrects the observed agreement for the agreement expected by chance. Landis and Koch [37] proposed the widely used verbal interpretation of κ values, according to which values up to 0.20 indicate slight agreement and values above 0.80 almost perfect agreement.

2.6 Online Learning for Imbalanced Data Streams

Online Bagging [38] is the online version of the bagging ensemble. In ordinary bagging each base learner is trained on a bootstrap sample of the data. In the online version, each arriving example is presented to each base learner k times, where k is drawn from a Poisson distribution with parameter $\lambda = 1$. Wang et al. [39] adapted this method to imbalanced streams. In their Oversampling Online Bagging (OOB), the parameter λ for an example of the minority class is increased according to the current ratio of the class sizes, so that minority examples are presented more often. The class sizes are tracked with a time-decayed average, which allows the method to follow changes in the imbalance.

ORB (Oversampling Rate Boosting) [12] extends OOB for JIT-SDP. It monitors the predictions that the model has made recently. If the model predicts one class much more often than expected, the resampling rate of the other class is increased further by a boosting factor. In this way the model is pushed back when it becomes biased towards one class, which happens frequently under verification latency.

It should be noted that in all these methods the resampling rate depends only on how many examples of each class have been observed. Whether the labels of these examples are correct plays no role. A commit that is wrongly labelled as defect-introducing is oversampled in the same way as a correctly labelled one. This property is one of the reasons why the effect of label noise on such learners deserves to be studied.

2.7 Learning with Noisy Labels

In the machine learning literature, several methods have been proposed for training classifiers on data with wrong labels. Natarajan et al. [40] considered the case where the probability that a label is flipped depends on the class, which is called class-conditional noise. They showed that, if the two flip rates are known, the loss function can be modified such that minimising it on the noisy data is equivalent in expectation to minimising the original loss on clean data. The correction contains the factor $1/(1 - \rho_0 - \rho_1)$, where ρ_0 and ρ_1 are the two flip rates, and becomes unstable when the sum of the rates approaches one. Co-teaching [41] trains two neural networks at the same time. Each network selects the examples with small loss in a mini-batch, which are likely to be correctly labelled, and passes them to the other network for training. Confident learning [42] estimates the joint distribution of the given and the true labels from the predicted probabilities of a classifier and uses it to identify and remove the examples that are probably mislabelled.

These methods were developed for the batch setting, in which the complete dataset is available and can be processed several times. They also assume that a wrong label stays wrong. In online JIT-SDP each example is seen once, its label arrives late, and a label

may be changed later from clean to defect-introducing. The methods can therefore not be applied directly, but the class-conditional noise model on which they are based is the one that is measured in the present work.

2.8 Summary

Table 2.2 summarises the studies that are most closely related to the present work and indicates in which respect each of them differs from it. The review shows that the accuracy of SZZ, the effect of label noise, time-aware evaluation and verification latency have each been studied, but mostly in isolation. The gaps that follow from this are stated in the next chapter.

Table 2.2: Summary of the most closely related studies.

Study	Focus	Difference from the present work
Kamei et al. [3]	Change metrics and models for JIT-SDP	Labels from SZZ are taken as correct; no time-aware evaluation
Da Costa et al. [8]	Framework for evaluating SZZ implementations	Indirect criteria; effect on prediction models not studied
Rosa et al. [9, 25]	Accuracy of SZZ variants against a developer-informed oracle	Oracle has no clean commits; effect on prediction models not studied
Herbold et al. [7]	Amount of tangling in bug-fixing commits	Line-level analysis; labels of commits and models not studied
Kim et al. [27]	Artificial label noise in file-level defect data	Noise is injected artificially; file-level prediction with batch evaluation
Fan et al. [29]	Effect of labels of four SZZ variants on JIT-SDP	RA-SZZ used as reference; no independent labels; no latency
Falessi et al. [10]	Order-preserving validation of defect classifiers	Label noise not considered
Cabral et al. [12]	Online JIT-SDP with verification latency and class imbalance evolution	SZZ labels taken as correct once they arrive
Zeng et al. [17]	Comparison of deep and simple JIT-SDP models	Label noise and latency not considered
Natarajan et al. [40]	Learning under class-conditional label noise	Batch setting; noise rates assumed to be known

Chapter 3

Research Gaps, Research Questions and Objectives

This chapter states the research gaps that were identified from the literature reviewed in Chapter 2. Based on these gaps, the research questions and the objectives of the thesis are formulated. The part of the objectives that is covered by the first stage of the work is indicated.

3.1 Research Gaps

The following gaps were identified.

Gap 1: The noise of SZZ labels has not been measured over complete project histories. The existing evaluations of SZZ either use indirect criteria [8] or check the algorithm on a sample of bug-fixing commits for which the introducing commit is known [9, 20]. These evaluations tell how often SZZ finds the right commit for a given fix. They do not tell how many of the commits in the history of a project receive a wrong label, which is the quantity that matters for a JIT-SDP model, because the model is trained on all commits. In particular, the rate of false positives among the clean commits and the rate of false negatives among the defect-introducing commits have not been reported separately for the different variants.

Gap 2: Models are tested against the same labels on which they are trained. In the JIT-SDP literature the test labels come from the same SZZ implementation as the training labels. The study that is closest to the present work [29] used one SZZ variant as the reference for the others. Hence it is not known how well a model that was trained on SZZ labels predicts labels that were obtained independently of the SZZ variant used for training.

Gap 3: The effect of label noise has not been separated from the effect of the evaluation procedure. It is known that random cross-validation gives optimistic results for time-ordered data [10, 11], and it is known that SZZ labels contain errors. These two factors are present together in most published results, and their individual contributions have not been measured in a common experimental design.

Gap 4: Label noise has not been studied under verification latency. The studies on online JIT-SDP [12, 31] model the delay of the labels but assume that the labels which finally arrive are correct. The studies on label noise use batch evaluation [27, 29]. The loss caused by SZZ noise for an online learner that receives delayed labels is therefore not known.

Gap 5: The way in which SZZ noise affects an online learner is not understood, and no remedy designed for this setting exists. Online learners for imbalanced streams oversample the minority class according to the number of positive labels they observe (Section 2.6), without regard to their correctness. Whether false positives or false negatives are more harmful for such learners is not known. The existing methods for learning with noisy labels assume a batch setting and stable labels [40, 42] and cannot be applied directly to a stream with delayed labels.

3.2 Research Questions

Based on these gaps, the thesis addresses the following research questions.

RQ1: How much do the labels of different SZZ variants disagree with one another and with independently obtained reference labels, and what type of error does each variant make? This question corresponds to Gap 1. To answer it, the labels of six SZZ variants are compared with reference labels on the same set of commits, and the false positive and false negative rates of each variant are estimated.

RQ2: How does the choice of the label source affect the measured performance of JIT-SDP models under different evaluation settings? This question corresponds to Gaps 2, 3 and 4. To answer it, models are trained on the reference labels and on the labels of each SZZ variant, and are evaluated with random cross-validation, with a chronological split and with prequential evaluation under verification latency. The predictions are scored against the reference labels and, for comparison, against the training labels.

RQ3: Through which mechanism does label noise reduce the performance of an online learner, and which of the two types of error is responsible? This question corresponds to the first part of Gap 5. It will be addressed in the next stage of the thesis.

RQ4: Can an online learner be modified so that it loses less performance when it is trained on SZZ labels, without access to clean labels and without losing performance when the labels are clean? This question corresponds to the second part of Gap 5. It will also be addressed in the next stage of the thesis.

3.3 Objectives

The objectives of the thesis are as follows.

1. To generate the labels of six SZZ variants for all projects of a dataset that provides independent reference labels, and to characterise the labelling errors of each variant in terms of false positive and false negative rates.
2. To examine the agreement between the SZZ variants and to identify the sources of the errors that the variants make.
3. To measure how much the evaluation procedure, namely the validation scheme and the labels used for scoring, changes the measured performance of JIT-SDP models.
4. To measure the loss in performance caused by training on SZZ labels when an online learner is evaluated on a commit stream with verification latency.
5. To identify the type of labelling error that is responsible for this loss and the mechanism through which it acts.
6. To design and evaluate a modification of the online learner that reduces this loss.

Objectives 1 to 4 belong to the first stage of the thesis and have been completed. They are reported in the following chapters. Objectives 5 and 6 belong to the next stage, and the plan for them is given in Section 7.3.

Chapter 4

Dataset

This chapter describes the dataset used in the study. It first states the requirements that the dataset has to satisfy and then describes the JIT-Defects4J dataset, the projects it contains, the reference labels, the change metrics and the data on the arrival time of the labels.

4.1 Requirements

The research questions place several requirements on the dataset.

- A label must be available for every commit of a project, including the clean commits, because JIT-SDP models are trained and tested on the complete commit history.
- The labels must not have been produced by the SZZ variant that is being evaluated. Otherwise the errors of that variant cannot be observed.
- The change metrics of the commits must be available, so that prediction models can be trained.
- The repositories of the projects must be accessible, because the SZZ variants have to be executed on them and the dates of the bug-fixing commits are needed for simulating verification latency.
- The dataset should contain several projects, so that the comparisons can be tested statistically across projects.

Most JIT-SDP datasets do not satisfy the second requirement. ApacheJIT [43], which contains 106,674 commits, and Defectors [44] are much larger than the dataset used here, but their labels were generated with SZZ. The developer-informed oracle of Rosa et al. [9] does not satisfy the first requirement, since it contains no clean commits. The JIT-Defects4J dataset satisfies all five requirements and was therefore selected.

4.2 The JIT-Defects4J Dataset

JIT-Defects4J was published by Ni et al. [45] together with their JIT-Fine approach. It was built from LLTC4J, the dataset of Herbold et al. [7], in which the changed lines of bug-fixing commits were labelled manually. JIT-Defects4J contains 27,319 commits from 21 open-source Java projects of the Apache Software Foundation. For every commit it provides the fourteen change metrics of Kamei et al. and a label that indicates whether the commit introduced a defect. The dataset was used as it was published. No project was added or removed, so the selection of projects could not be influenced by the results of this study.

4.3 Subject Projects

Table 4.1 lists the 21 projects. Sixteen of them are libraries of the Apache Commons family. The other five are a dependency manager (ant-ivy), a graph processing system (giraph), a data persistence framework (gora), a natural language processing toolkit (opennlp) and a columnar storage format (parquet-mr). The commits were made between 2001 and 2019. The length of the covered history of a project varies from about 6 years (parquet-mr) to about 18 years (commons-dbcp).

In total, 5,453 commits (20.0%) are marked as bug-fixing and 2,332 commits (8.54%) are defect-introducing according to the reference labels. The projects differ considerably in size and in the proportion of defect-introducing commits, as can be seen in Figure 4.1. The smallest project has 544 commits and the largest has 4,026. The number of defect-introducing commits ranges from 19 in commons-digester to 335 in commons-math, and the defect rate ranges from 1.8% to 19.3%. In every project the defect-introducing commits are a clear minority. Because of these differences, all comparisons in this work are made project by project and the results are then summarised over the projects (Section 5.9).

4.4 Reference Labels

The label provided with JIT-Defects4J is used in this work as the reference against which the SZZ variants and the predictions of the models are assessed. Since all later results depend on this label, its construction is described in detail.

According to the description given by the authors of the dataset [45], the labels were obtained in two steps. In the first step, they took the bug-fixing commits of LLTC4J. In LLTC4J every changed line of a bug-fixing commit was labelled by four participants, and a line is regarded as contributing to the bug fix only if at least three of them assigned this

Table 4.1: Projects of the JIT-Defects4J dataset. The last two columns give the number and the percentage of defect-introducing commits according to the reference labels.

Project	Period	Commits	Bug-fixing commits	Defect-introducing	
				Number	%
ant-ivy	2005–2018	1,771	761	332	18.7
commons-bcel	2001–2019	825	134	60	7.3
commons-beanutils	2001–2018	611	197	37	6.1
commons-codec	2003–2018	761	122	36	4.7
commons-collections	2001–2018	1,823	360	50	2.7
commons-compress	2003–2018	1,630	175	178	10.9
commons-configuration	2003–2018	1,838	358	155	8.4
commons-dbcp	2001–2019	1,037	348	58	5.6
commons-digester	2001–2017	1,079	221	19	1.8
commons-io	2002–2018	1,142	224	73	6.4
commons-jcs	2002–2018	831	237	88	10.6
commons-lang	2002–2018	2,969	491	146	4.9
commons-math	2003–2018	4,026	758	335	8.3
commons-net	2002–2018	1,121	197	117	10.4
commons-scxml	2005–2018	544	55	47	8.6
commons-validator	2002–2018	598	120	36	6.0
commons-vfs	2002–2018	1,110	201	114	10.3
giraph	2010–2018	844	118	163	19.3
gora	2010–2019	553	85	39	7.1
opennlp	2010–2018	1,086	83	91	8.4
parquet-mr	2012–2018	1,120	208	158	14.1
Total		27,319	5,453	2,332	8.54

label. In the second step, `git blame` was applied to each of these validated lines to find the commit that introduced it. The commits found in this way are the defect-introducing commits. All other commits are considered clean.

To make sure that the labels in the files used for the experiments are indeed these labels, the number of commits and of defect-introducing commits of every project was compared with the numbers reported in the paper. They agree for all 21 projects.

The reference labels are thus the result of the blame operation applied to manually validated bug-fixing lines. They are not a manual judgement about each commit. This has the following consequences for the interpretation of the results.

1. The reference labels and the SZZ labels differ in the lines from which the tracing starts. SZZ starts from all lines changed in a bug-fixing commit (after the filtering applied by the particular variant), whereas the reference starts only from the lines that were confirmed to be part of the fix. A comparison between the two therefore shows

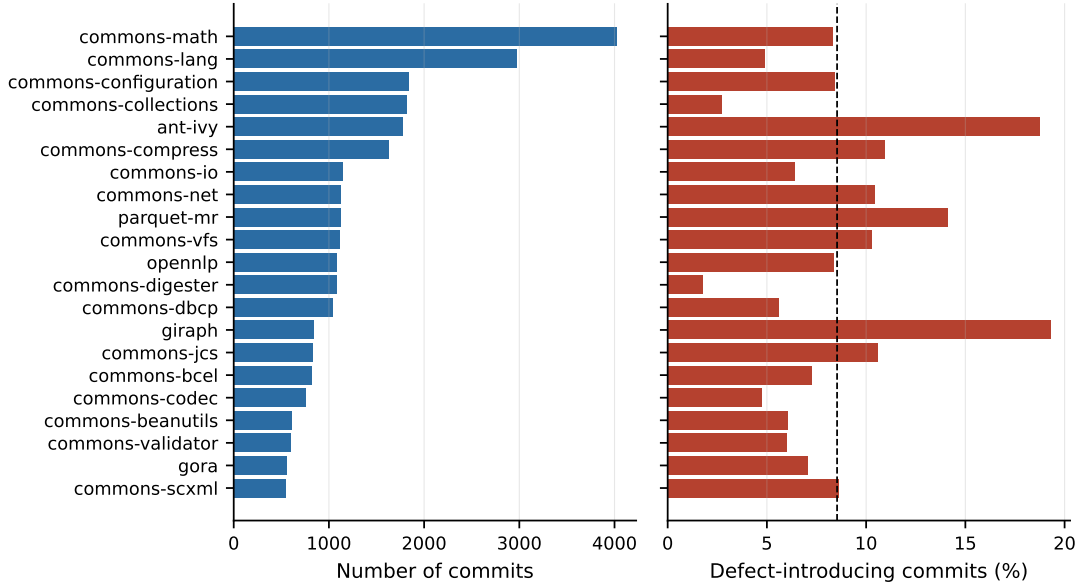


Figure 4.1: Number of commits and percentage of defect-introducing commits in each project. The dashed line marks the percentage in the whole dataset (8.54%).

the effect of tangled changes, and of the filters that the variants use to deal with them.

- Both use the blame operation to go from a line to a commit. Errors of the blame operation itself, for example when a line was last touched by a cosmetic change, are present in both and cannot be detected by comparing them. The amount of noise measured in this work should therefore be regarded as a lower bound of the true amount.
- Every commit that is defect-introducing according to the reference can be reached by blame from some line of a bug-fixing commit. When an SZZ variant misses such a commit, the reason cannot be that blame is unable to reach it. The reason must lie in the lines selected or in the filtering of the candidates. This fact is used in Section 6.1.4.
- The authors of the dataset excluded changes that do not add any line. Defects that are introduced only by deleting code are therefore not represented.

In the rest of this report the term *reference labels* is used for these labels. The term “ground truth” is avoided, because the labels are more reliable than SZZ labels with respect to tangling but are not free of error.

4.5 Change Metrics

Each commit is described by the fourteen change metrics proposed by Kamei et al. [3]. They are listed in Table 4.2. The metrics of the diffusion group measure how widely a change is spread over the code base, since changes that touch many parts are more difficult to understand. The size group measures how much code was changed. The purpose

group contains a single metric which indicates whether the change is a bug fix, since bug fixes are known to introduce new defects more often than other changes. The history group describes how the modified files were changed in the past, and the experience group describes how familiar the developer is with the project and with the modified subsystem.

Table 4.2: The fourteen change metrics used as features [3].

Group	Metric	Description
Diffusion	NS	Number of modified subsystems
	ND	Number of modified directories
	NF	Number of modified files
	Entropy	Distribution of the modified code across the files
Size	LA	Lines of code added
	LD	Lines of code deleted
	LT	Lines of code in the files before the change
Purpose	FIX	Whether the change is a defect fix
History	NDEV	Number of developers that changed the modified files
	AGE	Average time interval between the last and the current change
	NUC	Number of unique changes to the modified files
Experience	EXP	Number of changes made by the developer so far
	REXP	Recent experience of the developer (changes weighted by their age)
	SEXP	Experience of the developer on the modified subsystem

The values of the metrics were taken from the dataset as published and were not re-computed. In this way the features are identical for all label sources, and differences between the results obtained with different labels cannot be caused by differences in feature extraction. Several of the metrics have strongly skewed distributions. The transformations applied to them are part of the prediction models and are described in Section 5.4.

4.6 Label Sets

Besides the reference label, six further labels were attached to every commit, one for each SZZ variant of Table 2.1. They were generated in this work by running the variants on the repositories of the 21 projects, as described in Section 5.2. Each commit of the dataset therefore has seven binary labels. The data used in the experiments consist of one record per commit with the project name, the commit identifier, the author timestamp, the fourteen metrics, the seven labels and the label arrival times described in Section 4.8.

4.7 Data Preparation

The dataset is distributed in three files that correspond to the training, validation and test split used by its authors. These files were merged, since the present work uses its own evaluation settings. Duplicate records were removed on the combination of project and commit identifier. Within each project the commits were sorted by their author timestamp. This order defines the commit stream of the project and is used in all evaluation settings that depend on time.

The repositories of the 21 projects were cloned from GitHub for two purposes: to run the SZZ variants and to obtain the dates of the bug-fixing commits. The outputs of both steps were stored as small derived files, so that the experiments on the prediction models can be repeated without the repositories.

4.8 Label Arrival Times

To evaluate an online learner realistically, it must be known when the label of each commit would have become available. For a defect-introducing commit this is the time at which its defect was fixed, because only then the commit can be identified as the origin of the defect.

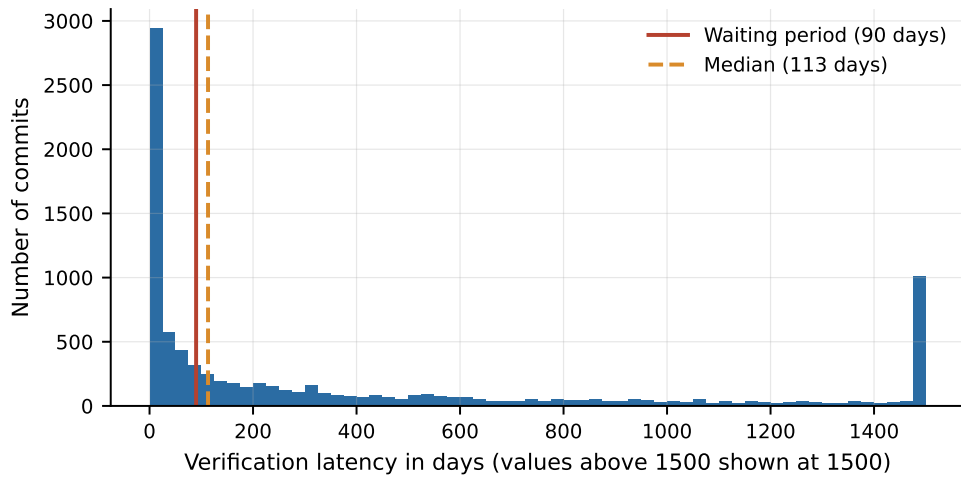
The arrival times were obtained as follows. The author date of each of the 5,453 bug-fixing commits was extracted from the repositories. Each SZZ variant produces a mapping from bug-fixing commits to the commits it considers defect-introducing. For a commit flagged by a variant, the arrival time of its label under that variant is the date of the earliest bug-fixing commit that is mapped to it. The verification latency of the commit is the difference between this date and the date of the commit itself.

Table 4.3 summarises the latencies over all commits that are linked to at least one bug-fixing commit, and Figure 4.2 shows their distribution. The distribution is strongly skewed. About one third of the labels arrive within 30 days, but the median is 113 days and one tenth of the labels need more than four years. With the waiting period of 90 days that is commonly used in online JIT-SDP [12], 53.0% of the defect labels arrive only after the waiting period has ended. As explained in Section 5.6, these commits are first presented to the learner as clean.

The reference labels need a separate treatment. The published dataset contains the label of each commit but not the link between a defect-introducing commit and the commit that fixed it. The arrival time of a reference label was therefore taken from the SZZ mappings: it is the date of the earliest bug-fixing commit that any of the six variants links to the commit. For 67.8% of the 2,332 defect-introducing commits such a link exists. For the remaining 32.2% no arrival time is available, and in the online evaluation their defect

Table 4.3: Verification latency of the commits that are linked to a bug-fixing commit.

Statistic	Value
Commits linked to a bug-fixing commit	8,819
Lower quartile of the latency	8 days
Median latency	113 days
Upper quartile of the latency	611 days
90th percentile of the latency	1,597 days
Labels arriving within 30 days	35.2%
Labels arriving after 90 days	53.0%
Labels arriving after one year	33.1%
Labels arriving after three years	16.2%

**Figure 4.2:** Distribution of the verification latency. A large part of the labels arrives after the waiting period of 90 days.

label never arrives.

Table 4.4 gives the latency separately for each label source. The medians of most sources lie between 77 and 110 days. R-SZZ is an exception with a median of only 26 days. This is a consequence of its selection rule, which keeps the most recent candidate commit and therefore prefers commits that are close in time to the fix.

The fact that the arrival times of the reference labels are derived from the SZZ mappings is a limitation of the study. The reference labels are independent of SZZ with respect to which commits are defect-introducing, but not with respect to when this becomes known. The possible influence of the incomplete coverage is examined in Section 6.2.4, and the limitation is discussed with the other threats to validity in Section 7.2.

Table 4.4: Availability of arrival times and latency of the positive labels of each label source.

Label source	Positive labels	With arrival time	Median latency	After 90 days
Reference	2,332	67.8%	109 days	52.8%
B-SZZ	8,060	100%	94 days	50.7%
AG-SZZ	5,876	100%	94 days	50.5%
MA-SZZ	6,154	100%	102 days	51.4%
L-SZZ	2,287	100%	77 days	48.9%
R-SZZ	3,015	100%	26 days	38.6%
RA-SZZ	5,553	100%	104 days	51.7%

4.9 Summary

The study uses the JIT-Defects4J dataset with 27,319 commits from 21 Apache Java projects, of which 8.54% are defect-introducing according to reference labels that were derived from manually validated bug-fixing lines. Each commit is described by fourteen change metrics and carries seven labels: the reference label and the labels of six SZZ variants. For the online evaluation, the arrival time of each label was reconstructed from the dates of the bug-fixing commits. More than half of the defect labels arrive later than the usual waiting period of 90 days.

Chapter 5

Methodology

This chapter describes the methodology of the first stage of the thesis. It begins with an overview of the approach. It then explains how the SZZ labels were generated and how their quality is analysed, describes the prediction models, the evaluation settings and the simulation of verification latency, and finally gives the performance measures, the statistical analysis and the experimental setup.

5.1 Overview of the Approach

Figure 5.1 shows the overall plan of the thesis and the part that is covered by the first stage. The first stage consists of two studies that build on each other.

In the first study, the *label quality analysis*, six SZZ variants are executed on the repositories of the 21 projects. Their labels are compared with the reference labels of the dataset. The result is a description of the errors of each variant, in particular its false positive and false negative rates, and a set of SZZ labels for every commit. This study answers RQ1.

In the second study, the *impact analysis*, prediction models are trained on the reference labels and on each of the six SZZ label sets. The models are evaluated in four evaluation settings, and their predictions are scored against the reference labels. This shows how much performance is lost when a model is trained on SZZ labels, and how much the evaluation setting itself changes the measured performance. This study answers RQ2.

The reference labels have two roles in this design. In the first study they are the standard against which the SZZ labels are assessed. In the second study they are the standard against which the predictions are scored, and they also provide the best available training labels, which serve as a baseline for the SZZ-trained models.

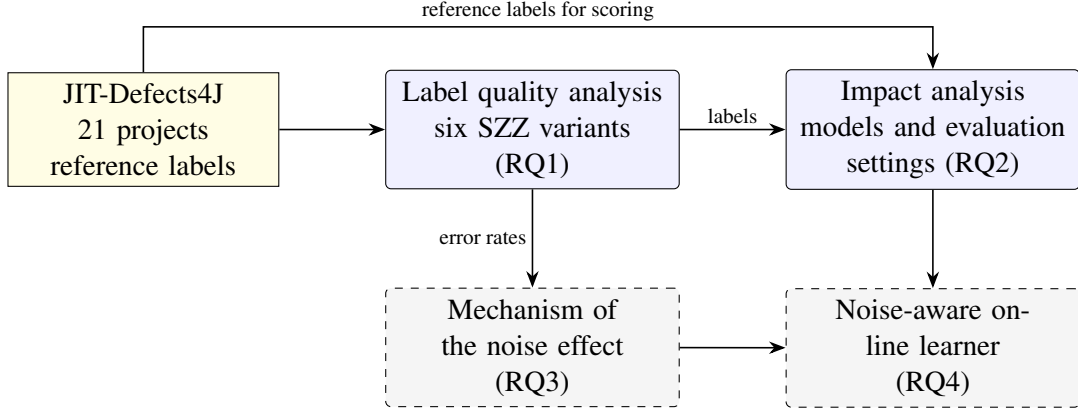


Figure 5.1: Overview of the thesis work. The boxes with solid borders belong to the first stage, which is reported here. The boxes with dashed borders are planned for the next stage.

5.2 Generation of SZZ Labels

The six SZZ variants of Table 2.1 were executed with PySZZ (version 2), the implementation released by Rosa et al. [25]. An existing implementation was preferred over writing a new one, because it has been used in earlier studies and its behaviour is documented. The configuration files supplied with the tool were used for each variant, with the following settings.

- The filter that removes candidate commits made after the bug report date was switched off for all variants, because the dataset does not contain the dates of the bug reports.
- For all variants except B-SZZ, commits that modify more than 20 files are ignored during the blame step. This is the default of the tool and is intended to skip very large commits such as mass reformatting.
- MA-SZZ, R-SZZ and L-SZZ use the options of `git blame` that detect lines moved within a file and lines moved or copied from other files changed in the same commit.
- RA-SZZ analyses only Java source files.

The input of the tool is a list of bug-fixing commits. The 5,453 commits of the dataset whose FIX metric is set were used for this purpose. The same list was given to all six variants, so that the differences between the label sets come only from the variants and not from the selection of the bug-fixing commits.

For every bug-fixing commit the tool returns the list of commits that the variant considers defect-introducing. These mappings were stored. A commit was then labelled as defect-introducing by a variant if it appears in the list of at least one bug-fixing commit, and as clean otherwise. The labels were joined with the dataset on the commit identifier. All 27,319 commits of the dataset received a label from each variant. Using the same set

of commits for every variant is important, because precision and recall are not comparable between variants when they are computed on different subsets.

5.3 Analysis of Label Quality

5.3.1 Comparison with the reference labels

For each variant, the SZZ label of every commit is compared with its reference label. The four possible outcomes are shown in Table 5.1. A commit that is flagged by the variant and is defect-introducing according to the reference is a true positive (TP). A commit that is flagged by the variant but is clean according to the reference is a false positive (FP). A commit that is not flagged but is defect-introducing according to the reference is a false negative (FN), and a commit that is clean according to both is a true negative (TN).

Table 5.1: Outcomes of comparing an SZZ label with the reference label.

SZZ label	Reference label	
	Defect-introducing	Clean
Defect-introducing	TP	FP
Clean	FN	TN

From these counts the following measures are computed:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (5.1)$$

Precision is the fraction of the commits flagged by the variant that are defect-introducing according to the reference. Recall is the fraction of the reference defect-introducing commits that the variant finds. In addition, MCC and Cohen’s κ between the SZZ labels and the reference labels are reported. Both are defined in Section 5.8.

5.3.2 Noise rates

For the study of label noise it is useful to describe a variant by the probability with which it changes a reference label. Two rates are defined:

$$\rho_0 = \frac{FP}{FP + TN}, \quad \rho_1 = \frac{FN}{FN + TP}. \quad (5.2)$$

The rate ρ_0 is the fraction of the clean commits that the variant labels as defect-introducing. It is called the false-alarm rate in this report. The rate ρ_1 is the fraction of the defect-introducing commits that the variant labels as clean. It is called the miss

rate and equals one minus the recall. A noise process that is described by these two rates is known as class-conditional noise [40]. If both rates were equal, the noise would be symmetric. Describing each variant by the pair (ρ_0, ρ_1) shows whether its noise is symmetric and, if not, which type of error dominates.

Since the two classes have very different sizes, a rate and the corresponding number of wrong labels must be distinguished. The clean class is about eleven times larger than the defect-introducing class, so a small value of ρ_0 can correspond to a larger number of wrong labels than a large value of ρ_1 . Both the rates and the counts are therefore reported.

5.3.3 Agreement between variants

The agreement between two label sets is measured with Cohen’s κ [36],

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad (5.3)$$

where p_o is the observed fraction of commits on which the two label sets agree and p_e is the fraction of agreement expected by chance, computed from the proportions of positive labels in the two sets. The value is computed for every pair of variants and for every variant with the reference labels. If the variants made their errors independently of each other, their mutual agreement would not be much higher than their agreement with the reference. A much higher mutual agreement indicates that the variants make the same errors.

5.3.4 Variation across projects and sources of false negatives

All measures are computed for the whole dataset and also for each project separately, to see how much the quality of the labels varies between projects.

To understand why the variants miss defect-introducing commits, the false negatives of each refined variant are divided into two groups by comparing them with B-SZZ. B-SZZ applies no filtering, so it traces every line changed by the bug-fixing commits. A false negative of a refined variant that is also a false negative of B-SZZ was missed already before any filtering. A false negative of a refined variant that B-SZZ labels correctly was found by the basic procedure and was then removed by the additional rules of the variant. The sizes of the two groups show how much of the miss rate of a variant is caused by its own filtering.

5.4 Prediction Models

Three prediction models are used in the impact analysis. They were chosen to cover different levels of model complexity and both batch and online learning. All three use the same fourteen change metrics, so that differences between them are not caused by different inputs.

5.4.1 LAPredict

LAPredict [17] is a logistic regression model that uses a single feature, the number of added lines (LA). In this work the feature is transformed as $\log(1 + \text{LA})$ and standardised, and the logistic regression is trained with class weights that are inversely proportional to the class frequencies, so that both classes contribute equally to the loss. A commit is predicted as defect-introducing when the predicted probability is at least 0.5. The model represents a very simple learner with only two parameters.

5.4.2 Random forest model based on JITLine

The second model follows the design of JITLine [18]: a random forest classifier combined with oversampling of the minority class. A random forest [46] with 100 trees is trained on the fourteen change metrics, which are transformed as $\text{sign}(x) \log(1 + |x|)$ to reduce their skewness. Before training, the minority class of the training data is oversampled with SMOTE [47] until both classes have the same size. SMOTE creates new minority examples by interpolating between an existing minority example and one of its nearest minority neighbours (five neighbours are used).

A random forest produces a probability for each commit, and a decision threshold is needed to turn it into a class. The threshold is chosen on the training data so that it maximises the G-mean (Section 5.8). To obtain probabilities that are not biased by the model having seen the same commits, the training data are divided into four consecutive blocks in time order. A model trained on the first block predicts the second block, a model trained on the first two blocks predicts the third, and so on. The threshold is selected on these predictions, and the final model is then trained on the complete training data.

The original JITLine additionally uses features derived from the tokens of the changed code. These are not used here, so that all three models work with the same features. For brevity this model is referred to as JITLine in the rest of the report. It represents a learner with high capacity, which can fit complex patterns in the training data.

5.4.3 ORB

The third model is ORB [12], an online learner that was designed for JIT-SDP with verification latency. It is an ensemble that is updated with one training example at a time. The implementation used in this work has 20 base learners. Each base learner is a logistic regression model that is trained by stochastic gradient descent. The features are transformed in the same way as for the random forest and are standardised with running estimates of their mean and variance.

When a training example (x, y) arrives, each base learner is updated k times with it, where k is drawn from a Poisson distribution with parameter λ . The value of λ is what distinguishes the methods of this family. In Online Bagging $\lambda = 1$. In OOB [39], λ depends on the class of the example and on the current proportion r_1 of positive examples, which is tracked as a decayed average,

$$r_1 \leftarrow \delta r_1 + (1 - \delta) I(y = 1), \quad \delta = 0.99, \quad (5.4)$$

where $I(y = 1)$ is 1 if the example is positive and 0 otherwise. The rate is then

$$\lambda_{\text{OOB}} = \begin{cases} (1 - r_1)/r_1 & \text{if } y = 1 \text{ and } r_1 < 0.5, \\ r_1/(1 - r_1) & \text{if } y = 0 \text{ and } r_1 > 0.5, \\ 1 & \text{otherwise.} \end{cases} \quad (5.5)$$

For example, if 10% of the recent training examples are positive, a positive example is presented on average nine times to each base learner.

ORB multiplies this rate by a boosting factor b that depends on the recent predictions of the model. Let $\bar{p}$ be a moving average of the predictions (1 for defect-introducing, 0 for clean), which is updated after every prediction as $\bar{p} \leftarrow \theta \bar{p} + (1 - \theta) \hat{y}$ with $\theta = 0.4$. If the model predicts the positive class less often than the proportion of positive examples it has seen ($\bar{p} < r_1$), positive training examples are boosted. If it predicts the positive class more often ($\bar{p} > r_1$), negative training examples are boosted:

$$b = \begin{cases} l_0 \frac{m^{\bar{p}} - m^{r_1}}{1 - m^{r_1}} + 1 & \text{if } y = 1 \text{ and } \bar{p} < r_1, \\ l_1 \frac{n^{1-\bar{p}} - n^{1-r_1}}{1 - n^{1-r_1}} + 1 & \text{if } y = 0 \text{ and } \bar{p} > r_1, \\ 1 & \text{otherwise,} \end{cases} \quad (5.6)$$

with the parameter values $l_0 = 10$, $l_1 = 12$, $m = 1.5$ and $n = 3$ proposed by the authors of ORB [12]. The final rate is $\lambda = \lambda_{\text{OOB}} \cdot b$. A commit is predicted as defect-introducing when the majority of the base learners predict it so.

It can be seen from Equations 5.5 and 5.6 that the amount of oversampling depends on the number of positive labels received and on the predictions of the model, but not on whether the labels are correct. ORB represents the online learners that are used when the commit stream is processed in a realistic way.

5.5 Evaluation Settings

The models are evaluated in four settings, which are illustrated in Figure 5.2. They are ordered from the least to the most realistic.

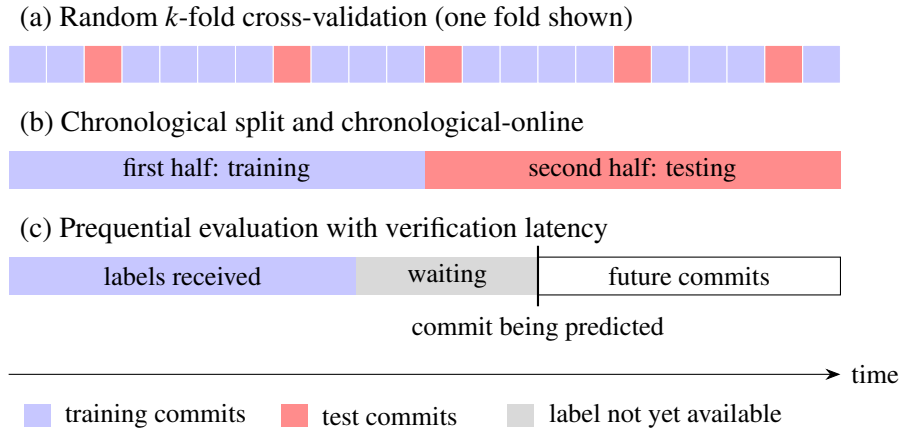


Figure 5.2: The evaluation settings. The commits of a project are ordered by time from left to right.

Random k -fold cross-validation. The commits of a project are divided at random into ten folds, stratified by the training label. Each fold is used once for testing while the model is trained on the other nine. The order of the commits is ignored, so the model is trained on commits that are both older and newer than the test commits. This setting is included because it is widely used in the literature. It is applied to the two batch models.

Chronological split. The commits of a project are ordered by time. The model is trained on the first half and tested on the second half. The model never sees commits from the future, but it is assumed that the labels of all training commits are known at the time of training. This setting is also applied to the two batch models.

Chronological-online. This setting uses the same split for ORB. The online learner processes the commits of the first half one by one with their labels and is then kept fixed while it predicts the second half. It makes it possible to compare ORB with the batch models on the same test commits, and to see how ORB behaves when it is not updated during testing.

Prequential evaluation with verification latency. The commits are processed in time order. Each commit is first predicted by the current model. Its label is given to the model for training only at the time at which it would have become available in practice. The details are given in the next section. The model is updated continuously over the whole stream. This setting is the closest to the actual use of a JIT-SDP model and is applied to ORB.

5.6 Simulation of Verification Latency

In the prequential setting the labels are delivered to the learner according to the procedure of Cabral et al. [12] with a waiting period of $W = 90$ days. Let t be the time of a commit. Four cases are distinguished according to the training label of the commit and the time of the bug-fixing commit linked to it (Section 4.8). They are shown in Figure 5.3.

1. The commit is clean according to the training labels. Nothing is known about it until the waiting period has passed. At time $t + W$ it is given to the learner as a clean example.
2. The commit is defect-introducing and its fix is made before $t + W$. It is given to the learner as a defect-introducing example at the time of the fix.
3. The commit is defect-introducing and its fix is made after $t + W$. At time $t + W$ no defect is known yet, so the commit is given to the learner as a clean example. At the time of the fix it is given again, this time as a defect-introducing example.
4. The commit is defect-introducing according to the training labels, but no fix date is available for it. It is given to the learner as a clean example at $t + W$ and is never corrected.

Cases 3 and 4 are the ones in which the learner is trained on a wrong label because of the delay. In case 3 the error is temporary. In case 4 it is permanent. These errors occur with every label source, including the reference labels, and they concern only defect-introducing commits. They add to whatever errors the labels themselves contain.

The prediction for a commit is made at time t and is scored at once, using the label against which the model is evaluated (Section 5.7). The scoring is not delayed, because its purpose is to measure the quality of the predictions and not to simulate what a user of the model would know. Algorithm 1 summarises the procedure.

5.7 Scoring of the Predictions

The predictions of a model can be compared with two different labels.

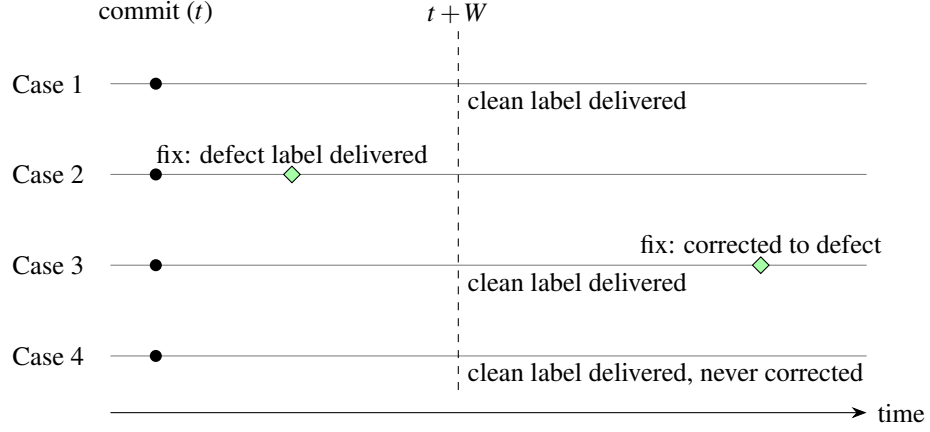


Figure 5.3: Delivery of training labels under verification latency. In case 3 the learner first receives a wrong clean label, which is corrected when the defect is fixed.

Algorithm 1 Prequential evaluation with verification latency

Require: commits $c_1, \dots, c_N$ of a project in time order, with features x_i , time t_i , training label y_i , evaluation label e_i and fix time f_i (if available); waiting period W ; online learner M

```

1:  $Q \leftarrow$  empty priority queue of pending training examples, ordered by delivery time
2: for  $i = 1$  to  $N$  do
3:   while  $Q$  is not empty and the earliest delivery time in  $Q$  is at most  $t_i$  do
4:     remove the earliest example  $(x_j, y)$  from  $Q$  and update  $M$  with it
5:   end while
6:    $\hat{y}_i \leftarrow$  prediction of  $M$  for  $x_i$ 
7:   update the evaluation measures with  $(e_i, \hat{y}_i)$ 
8:   if  $y_i = 1$  and  $f_i$  is available then
9:     if  $f_i \leq t_i + W$  then
10:      add  $(x_i, 1)$  to  $Q$  with delivery time  $f_i$ 
11:     else
12:      add  $(x_i, 0)$  to  $Q$  with delivery time  $t_i + W$ 
13:      add  $(x_i, 1)$  to  $Q$  with delivery time  $f_i$ 
14:     end if
15:   else
16:     add  $(x_i, 0)$  to  $Q$  with delivery time  $t_i + W$ 
17:   end if
18: end for

```

Scoring against the reference labels. The predictions are compared with the reference labels of the test commits. These labels are never used for training when the model is trained on SZZ labels. The result shows how well the model identifies the commits that are defect-introducing according to the best available standard. This is the main way of scoring in this work, and it is used unless stated otherwise.

Scoring against the training labels. The predictions are compared with the labels of the same SZZ variant that was used for training. This is what is done in the literature, where only one set of labels is available. It is called *self-scoring* in this report.

For a model trained on SZZ labels both scores are computed from the same predictions. The difference between them shows how much the usual way of scoring overestimates or underestimates the performance with respect to the reference. For a model trained on the reference labels the two scores are the same by definition.

5.8 Performance Measures

5.8.1 MCC and G-mean

The main performance measure is the Matthews correlation coefficient [34],

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}, \quad (5.7)$$

where the four counts are now obtained by comparing the predictions with the evaluation labels. MCC ranges from -1 to $+1$. It is 0 for a model that predicts at random or assigns all commits to one class, and it is high only if the model performs well on both classes. It was chosen because of the class imbalance: a model that predicts every commit as clean would reach an accuracy of 91.5% on this dataset but has an MCC of 0. MCC has also been recommended for defect prediction in place of the F-measure [33, 35].

As a second measure the G-mean is reported, which is the geometric mean of the recall on the two classes,

$$\text{G-mean} = \sqrt{\frac{TP}{TP + FN} \cdot \frac{TN}{TN + FP}}. \quad (5.8)$$

It is commonly used in online JIT-SDP [12] and becomes zero when one of the classes is not recognised at all.

5.8.2 Measures in the prequential setting

In the prequential setting the performance is not a single value, since the model changes during the stream. The usual practice is to compute the measures from a confusion matrix with a fading factor [30]. Before each update, the four counts are multiplied by a factor $f < 1$, and then the count that corresponds to the new prediction is increased by one. Older predictions thus have less weight. With $f = 0.99$, which is used here, the weights add up to $1/(1 - f) = 100$, so the matrix effectively describes the last hundred commits.

Let MCC_i be the MCC computed from the faded matrix after the i -th commit of a stream of N commits. Two summaries of this sequence are used.

- The *terminal* value MCC_N is the value at the end of the stream. It reflects the performance on roughly the last hundred commits.
- The *time-averaged* value $\frac{1}{N} \sum_{i=1}^N MCC_i$ is the mean over the whole stream.

The terminal value depends on a small part of the stream, which contains on average only about eight defect-introducing commits. It therefore varies strongly from project to project. The standard deviation of the project-level values is 0.092 for the terminal value and 0.051 for the time-averaged value. Because it is more stable, the time-averaged value is used as the primary summary in this work, and the terminal value is reported as a secondary one. The convergence result of Gama et al. for the faded error rate does not carry over to MCC, which is not a sum of per-example losses, so neither summary can claim to be the standard one. All results of the prequential setting are therefore given for both summaries, and the sensitivity of the conclusions to the fading factor is examined separately (Section 6.2.8).

5.9 Statistical Analysis

5.9.1 Unit of analysis

Each configuration is run with ten random seeds on each of the 21 projects. The seeds affect the random forest, the oversampling, the assignment of commits to folds and the Poisson draws of ORB. Runs with different seeds on the same project are not independent observations, because they use the same data. For every configuration the results of the ten seeds are therefore averaged for each project first, and the statistical tests are carried out on the 21 project-level values. All comparisons are paired by project: the two conditions that are compared are always evaluated on the same projects.

5.9.2 Test and effect sizes

Let $d_1, \dots, d_{21}$ be the differences between two conditions for the 21 projects. The following quantities are reported for each comparison.

Wilcoxon signed-rank test. The null hypothesis that the differences are symmetric around zero is tested with the Wilcoxon signed-rank test [48]. The absolute differences are ranked, and the sum W^+ of the ranks of the positive differences and the sum W^- of the ranks of the negative differences are computed. The test is non-parametric and does not assume that the differences are normally distributed, which cannot be assumed for MCC values of 21 projects.

Rank-biserial correlation. As a measure of how consistently the projects point in the same direction, the matched-pairs rank-biserial correlation is used,

$$r = \frac{W^+ - W^-}{W^+ + W^-}. \quad (5.9)$$

It is $+1$ if all differences are positive, -1 if all are negative, and 0 if the positive and negative ranks balance. Values of 0.1 , 0.3 and 0.5 are commonly regarded as small, medium and large.

Hodges–Lehmann estimate. The size of the difference is estimated with the Hodges–Lehmann estimator [49], which is the median of all pairwise averages $(d_i + d_j)/2$ with $i \leq j$. It is the estimate of location that belongs to the signed-rank test and is less sensitive to single extreme projects than the mean.

Confidence interval. A 95% confidence interval for the Hodges–Lehmann estimate is obtained with the percentile bootstrap [50]. The 21 projects are resampled with replacement 10,000 times, the estimate is recomputed for each resample, and the 2.5th and 97.5th percentiles of the resulting values are taken as the limits.

5.9.3 Correction for multiple comparisons

In total 74 tests are carried out in the impact analysis. They belong to the five families listed in Table 5.2. When many tests are made, some of them are expected to be significant by chance. The p -values are therefore adjusted with the method of Holm [51]. For m tests with ordered p -values $p_{(1)} \leq \dots \leq p_{(m)}$, the adjusted value of the i -th one is

$$\tilde{p}_{(i)} = \max_{j \leq i} \min(1, (m - j + 1) p_{(j)}). \quad (5.10)$$

The correction is applied in two ways. First, it is applied within each family of related tests. Second, it is applied to all 74 tests together. The second correction is stricter. It is reported because the families were defined during the analysis and not before it, so that it could be argued that the grouping was chosen favourably. A result that remains significant after the correction over all tests does not depend on how the families were formed. The main conclusions of this report are based on this global correction. A significance level of 0.05 is used.

Table 5.2: Families of statistical tests in the impact analysis.

Family	Comparison	Tests
Evaluation setting	Random k -fold against chronological split, for each batch model and label source	14
Scoring	Self-scored against reference-scored, for each model, SZZ variant and evaluation setting (for the prequential setting with both summaries)	42
Label source	ORB trained on reference labels against ORB trained on each SZZ variant, prequential setting (both summaries)	12
Learner	Each batch model against ORB on the chronological split	4
Online setting	ORB in the chronological-online setting against ORB in the prequential setting	2
Total		74

5.10 Additional Analyses

Besides the main experiment, some additional analyses were carried out to examine specific questions that arose from the results. Their design is described here, and their results are given in Chapter 6.

Predictability of SZZ errors. To examine whether the false positives of SZZ have properties that a model can learn, a classifier is trained to separate the false positives of B-SZZ from its true positives. Only the 8,060 commits flagged by B-SZZ are used. For each project they are split chronologically into two halves, and a random forest with 100 trees is trained on the first half with the fourteen change metrics. Its area under the ROC curve (AUC) on the second half is compared with the AUC of the same procedure when the training labels are randomly permuted. Five seeds are used.

Separation of label delay and model adaptation. The chronological-online and prequential settings differ in two respects: whether the labels are delayed, and whether the model continues to learn during testing. To separate the two, ORB is evaluated in all four

combinations of these factors with reference labels: frozen or adaptive learner, and immediate or delayed labels. In all four cases the performance is computed on the second half of the stream with the ordinary (not faded) MCC, so that the cells are directly comparable.

Sensitivity to the prequential summary. The comparison of label sources in the prequential setting is repeated for five values of the fading factor (0.90, 0.95, 0.99, 0.995 and 0.999) and for four lengths of an initial warm-up period that is excluded from the average (0%, 5%, 10% and 25% of the stream).

Availability of arrival times. For the reference labels, an arrival time is available for only 67.8% of the defect-introducing commits (Section 4.8), whereas it is available for all commits flagged by the SZZ variants. To check whether this difference influences the comparison, the missing arrival times of the reference labels are imputed by drawing latencies at random from the observed latency distribution, and the evaluation is repeated.

Exclusion of small projects. Some projects contain few defect-introducing commits. The main comparisons are repeated after excluding the projects with fewer than 25 defect-introducing commits in the second half of their history, which leaves 13 projects.

5.11 Experimental Setup

The experiment of the impact analysis combines the three models with the seven label sources, the evaluation settings applicable to each model and the two ways of scoring. Table 5.3 summarises the factors. For each project and seed there are 78 configurations: each of the two batch models is evaluated in two settings with seven label sources scored against the reference and six scored against the training labels, and the same holds for ORB with its two settings. With 21 projects and ten seeds this gives $78 \times 21 \times 10 = 16,380$ evaluation runs.

Table 5.3: Factors of the experiment in the impact analysis.

Factor	Levels
Model	LApredict, JITLine (batch models); ORB (online model)
Training labels	Reference labels; labels of B-SZZ, AG-SZZ, MA-SZZ, L-SZZ, R-SZZ and RA-SZZ
Evaluation setting	Random 10-fold cross-validation and chronological split (batch models); chronological-online and prequential with verification latency (ORB)
Scoring	Against the reference labels; against the training labels
Projects	21
Random seeds	10

The experiments were implemented in Python 3.13 with scikit-learn 1.9.0 for the batch models, imbalanced-learn 0.14.2 for SMOTE, and NumPy, pandas and SciPy for the data handling and the statistical tests. The online learner and the prequential evaluation were implemented for this work. The versions of all libraries are fixed in the project, so that the results can be regenerated exactly.

Two measures were taken to keep the results consistent. First, every table and figure in this report is produced by a script from the stored outputs of the experiments. Second, before the experiments of the impact analysis are run, a check recomputes the counts of Table 5.1 from the label columns of the experimental data and compares them with the results of the label quality analysis. This ensures that the models are trained on exactly the labels whose quality is reported.

5.12 Summary

In the first study, six SZZ variants are run with PySZZ on 5,453 bug-fixing commits, and their labels are compared with the reference labels in terms of precision, recall, noise rates and agreement. In the second study, three models of different complexity are trained on seven label sources and evaluated in four settings, from random cross-validation to prequential evaluation with verification latency. The predictions are scored against the reference labels and against the training labels. The comparisons are tested at the level of projects with the Wilcoxon signed-rank test, and the p -values are corrected for multiple comparisons.

Chapter 6

Results and Discussion

This chapter presents the results of the first stage of the thesis. Section 6.1 gives the results of the label quality analysis and answers RQ1. Section 6.2 gives the results of the impact analysis and answers RQ2. Section 6.3 summarises the findings.

6.1 Quality of the SZZ Labels

6.1.1 Overall quality

Table 6.1 shows, for each variant, the number of commits it flags as defect-introducing and the outcome of the comparison with the reference labels. Table 6.2 gives the measures computed from these counts. All numbers refer to the same 27,319 commits, of which 2,332 are defect-introducing and 24,987 are clean according to the reference.

Table 6.1: Number of commits flagged by each SZZ variant and comparison with the reference labels.

Variant	Flagged	% of commits	TP	FP	FN	TN
B-SZZ	8,060	29.5	1,495	6,565	837	18,422
AG-SZZ	5,876	21.5	1,090	4,786	1,242	20,201
MA-SZZ	6,154	22.5	1,124	5,030	1,208	19,957
L-SZZ	2,287	8.4	622	1,665	1,710	23,322
R-SZZ	3,015	11.0	699	2,316	1,633	22,671
RA-SZZ	5,553	20.3	1,022	4,531	1,310	20,456

The following observations can be made from these tables and from Figure 6.1.

The precision of all variants is low. The precision lies between 0.183 and 0.272. This means that between 72.8% and 81.7% of the commits that a variant labels as defect-introducing are clean according to the reference. B-SZZ flags 8,060 commits, which is

Table 6.2: Quality of the SZZ labels with respect to the reference labels. The best value in each column is shown in bold.

Variant	Precision	Recall	F_1	MCC	κ	ρ_0	ρ_1
B-SZZ	0.186	0.641	0.288	0.232	0.179	0.263	0.359
AG-SZZ	0.186	0.467	0.266	0.188	0.163	0.192	0.533
MA-SZZ	0.183	0.482	0.265	0.188	0.161	0.201	0.518
L-SZZ	0.272	0.267	0.269	0.202	0.202	0.067	0.733
R-SZZ	0.232	0.300	0.262	0.185	0.183	0.093	0.700
RA-SZZ	0.184	0.438	0.259	0.178	0.158	0.181	0.562

29.5% of all commits, whereas only 8.54% of the commits are defect-introducing according to the reference. The refined variants flag fewer commits, but their precision is not higher than that of B-SZZ, with the exception of L-SZZ and R-SZZ, which keep only one candidate per bug-fixing commit.

The recall decreases with every refinement. B-SZZ finds 64.1% of the reference defect-introducing commits. AG-SZZ and MA-SZZ find about 47% to 48%, RA-SZZ 43.8%, R-SZZ 30.0% and L-SZZ only 26.7%. The two variants with the highest precision are thus the ones that miss the most defects.

The two types of error are not balanced, and the balance differs between the variants. Figure 6.1(b) plots the false-alarm rate ρ_0 against the miss rate ρ_1 . The variants lie along a line from the upper left to the lower right. B-SZZ has the highest false-alarm rate (0.263) and the lowest miss rate (0.359). L-SZZ and R-SZZ have low false-alarm rates (0.067 and 0.093) and very high miss rates (0.733 and 0.700). AG-SZZ, MA-SZZ and RA-SZZ lie in between. In no case are the two rates equal, so the noise of SZZ cannot be described as symmetric. The noise also cannot be described by a single pair of rates for SZZ in general, because the pair depends strongly on the variant.

No variant is better than the others on all measures. B-SZZ has the highest recall, F_1 and MCC. L-SZZ has the highest precision and κ . The F_1 values of the six variants are almost the same (0.259 to 0.288), although the variants behave very differently. A comparison of SZZ variants on a single summary measure can therefore be misleading.

Figure 6.2 shows the same results as numbers of commits. Because the clean class is much larger than the defect-introducing class, the number of false positives exceeds the number of false negatives for five of the six variants, even for those whose miss rate is much higher than their false-alarm rate. For B-SZZ there are 6,565 false positives and 837 false negatives, a ratio of about eight to one. Only for L-SZZ are the two numbers approximately equal (1,665 and 1,710). When the effect of the two types of error on a

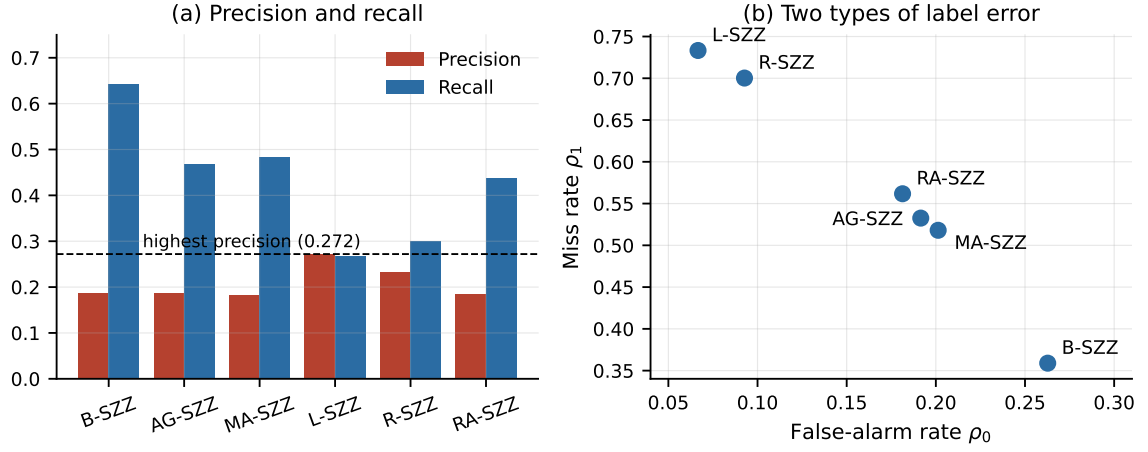


Figure 6.1: (a) Precision and recall of the SZZ variants with respect to the reference labels. (b) False-alarm rate and miss rate of the variants.

learner is studied, it will therefore be necessary to distinguish between the effect of a single wrong label of each type and the effect of their different numbers.

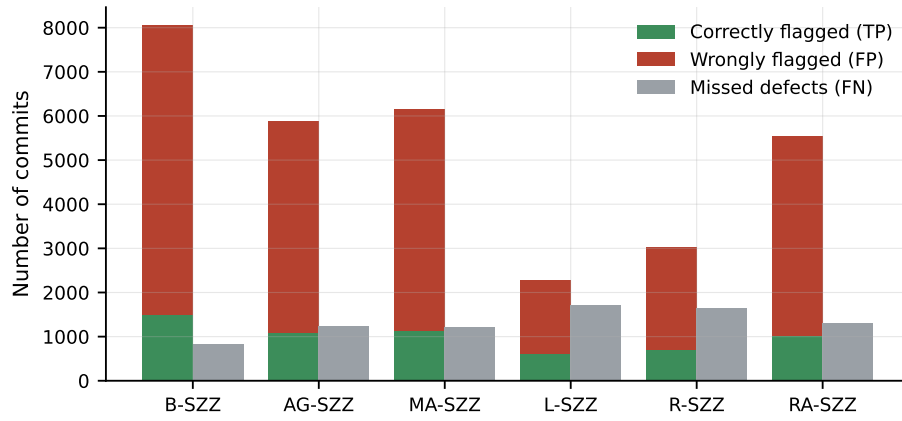


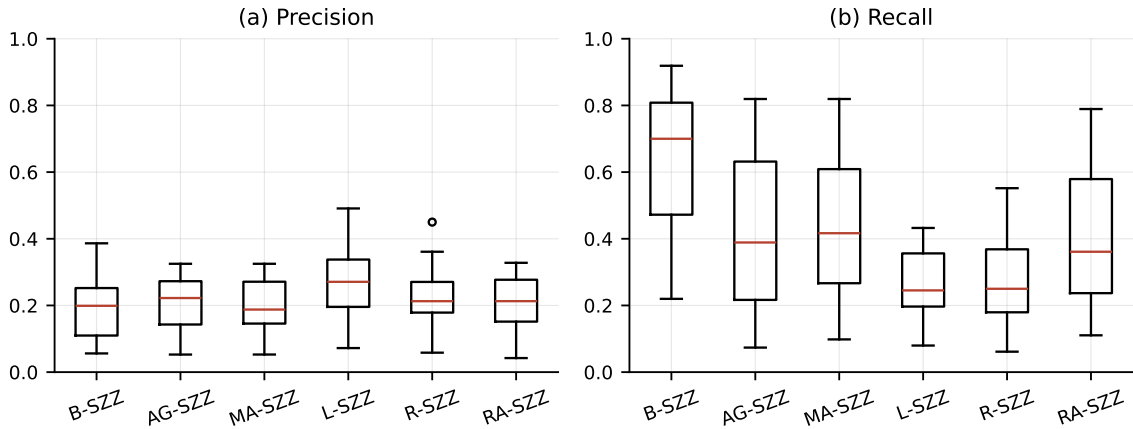
Figure 6.2: Number of correctly flagged, wrongly flagged and missed commits for each variant.

6.1.2 Variation across projects

Table 6.3 gives the minimum, median and maximum of precision and recall over the 21 projects, and Figure 6.3 shows the distributions. The quality of the labels varies widely between projects. The recall of B-SZZ is 0.220 in *opennlp* and 0.919 in *commons-beanutils*. Its precision is 0.056 in *commons-collections* and 0.386 in *ant-ivy*. For AG-SZZ the recall ranges from 0.074 to 0.819. The values for the whole dataset in Table 6.2 are thus not representative of an individual project. This is one of the reasons why the statistical comparisons in the impact analysis are made per project.

Table 6.3: Minimum, median and maximum of precision and recall over the 21 projects.

Variant	Precision			Recall		
	Min.	Median	Max.	Min.	Median	Max.
B-SZZ	0.056	0.199	0.386	0.220	0.700	0.919
AG-SZZ	0.053	0.222	0.325	0.074	0.389	0.819
MA-SZZ	0.053	0.188	0.325	0.098	0.417	0.819
L-SZZ	0.072	0.271	0.491	0.080	0.245	0.432
R-SZZ	0.058	0.213	0.450	0.061	0.250	0.552
RA-SZZ	0.042	0.213	0.328	0.110	0.361	0.789

**Figure 6.3:** Distribution of precision and recall of each variant over the 21 projects.

6.1.3 Agreement between the variants

Figure 6.4 shows Cohen’s κ for every pair of label sets. The first row and column give the agreement of each variant with the reference labels, and the remaining cells give the agreement between variants.

The agreement of the variants with the reference is between 0.158 and 0.202, which is slight agreement in the terms of Landis and Koch [37]. The agreement among the variants is much higher. AG-SZZ, MA-SZZ and RA-SZZ agree with each other at 0.86 to 0.93, which is almost perfect agreement. B-SZZ agrees with these three at 0.63 to 0.70. L-SZZ and R-SZZ agree with each other at 0.64 and with the others at 0.33 to 0.60. Two groups can be recognised: the variants that filter lines (AG-SZZ, MA-SZZ, RA-SZZ) and the variants that select a single candidate (L-SZZ, R-SZZ).

Even the lowest agreement between two variants (0.33) is higher than the highest agreement of any variant with the reference (0.20). The variants are therefore much more similar to each other than to the reference. This is plausible, since all of them start from the same bug-fixing commits and use the same blame mechanism, so they are affected by tangled changes in the same way.

This has a practical implication. When two SZZ variants or implementations produce

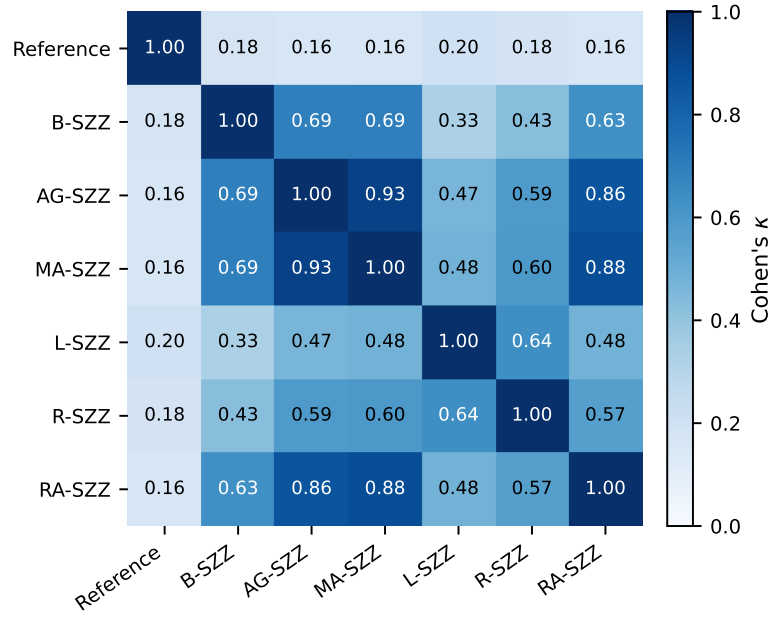


Figure 6.4: Cohen’s κ between the reference labels and the labels of the six SZZ variants.

similar labels, this is sometimes taken as an indication that the labels are reliable. The present result shows that this conclusion is not justified. The variants agree to a large extent on commits that are wrongly flagged. In total, 8,821 commits are flagged by at least one variant and 1,534 by all six. Of the 2,332 reference defect-introducing commits, 1,581 (67.8%) are flagged by at least one variant, only 450 (19.3%) are flagged by all six, and 751 (32.2%) are not flagged by any variant.

6.1.4 Sources of the false negatives

The high miss rates of the refined variants raise the question of where the missed commits are lost. As explained in Section 4.4, every reference defect-introducing commit can be reached by blame, so the losses cannot be explained by defects that leave no traceable line. Table 6.4 divides the false negatives of each refined variant into those that B-SZZ also misses and those that B-SZZ finds.

Two sources of roughly similar size can be seen.

The first source is common to all variants. B-SZZ itself misses 837 reference defect-introducing commits, and between 763 and 808 of them are also missed by each refined variant. These commits are not reached although B-SZZ traces every changed line of the bug-fixing commits. Since both procedures use blame, the difference must lie in the starting points of the tracing, that is, in the set of bug-fixing commits and in the lines from which the tracing starts. In this work the bug-fixing commits were taken from the FIX metric of the dataset, while the reference labels were derived from the validated bug-fixing commits of LLTC4J, and the two sets need not be identical. This part of the miss

Table 6.4: False negatives of the refined variants, divided according to whether B-SZZ also misses the commit. The last column gives the share of the 1,495 true positives of B-SZZ that the variant no longer flags.

Variant	False negatives	Also missed by B-SZZ	Found by B-SZZ but removed	Share of B-SZZ true positives removed
AG-SZZ	1,242	765 (62%)	477 (38%)	31.9%
MA-SZZ	1,208	763 (63%)	445 (37%)	29.8%
RA-SZZ	1,310	765 (58%)	545 (42%)	36.5%
R-SZZ	1,633	808 (49%)	825 (51%)	55.2%
L-SZZ	1,710	807 (47%)	903 (53%)	60.4%

rate can therefore not be attributed to the SZZ variants alone.

The second source is the filtering of the variants. Between 445 and 903 of the commits that B-SZZ labels correctly are removed again by the refined variants. AG-SZZ, MA-SZZ and RA-SZZ remove about 30% to 37% of the true positives of B-SZZ. R-SZZ and L-SZZ remove more than half of them (55.2% and 60.4%). For these two variants the filtering accounts for the larger part of the false negatives.

The refinements achieve what they were designed for: the false-alarm rate falls from 0.263 for B-SZZ to 0.067 for L-SZZ. However, this is obtained at the cost of removing a large number of correct labels. From B-SZZ to L-SZZ the number of false positives falls from 6,565 to 1,665, a reduction of 75%, but the number of true positives also falls, from 1,495 to 622, a reduction of 58%. Because wrong and correct labels are removed at similar rates, the precision improves only from 0.186 to 0.272.

It was also observed that the refined variants are not strict subsets of B-SZZ. Between 4.7% (R-SZZ) and 11.9% (RA-SZZ) of the commits they flag are not flagged by B-SZZ. The reason is that these variants use different blame options, for example the detection of moved lines, and can therefore arrive at different commits.

6.1.5 Discussion and answer to RQ1

RQ1 asked how much the SZZ variants disagree with one another and with the reference labels, and what type of error each variant makes.

With respect to the reference labels, all variants show a large amount of disagreement. At least 72.8% of the commits flagged by any variant are clean according to the reference, and at least 35.9% of the reference defect-introducing commits are missed. The agreement with the reference in terms of κ is at most 0.202. With respect to one another, the variants agree much more strongly, with κ up to 0.933.

The type of error depends on the design of the variant. B-SZZ, which traces all changed lines, mainly produces false positives. L-SZZ and R-SZZ, which keep a sin-

gle candidate, mainly miss defect-introducing commits. The variants that filter lines lie between these two. The successive refinements of SZZ have reduced the false positives but have increased the false negatives, so the errors have been moved from one type to the other more than they have been reduced.

The precision values measured here (0.18 to 0.27) are much lower than those reported by Rosa et al. [25] for the same implementation (about 0.39 to 0.66). The two results do not contradict each other, because they answer different questions. Rosa et al. consider bug-fixing commits for which an introducing commit is known and ask whether SZZ finds it. Here, all commits of a project are considered, most of which are clean, and it is asked whether the label of each commit is correct. A bug-fixing commit for which no validated fix line leads to an introducing commit still produces candidates in SZZ, and these candidates count as false positives in the present analysis but are outside the scope of the other one. In addition, as noted in Section 4.4, the reference labels used here are themselves derived with blame, and part of the disagreement comes from different starting points of the tracing. The numbers should be read as the disagreement between SZZ and labels derived from validated fix lines, and not as an absolute error rate of SZZ.

Two conclusions are important for the rest of the thesis. First, the label noise that a JIT-SDP model is exposed to is class-conditional and depends on the variant, so experiments with artificial symmetric noise would not represent it. Second, the measured pairs (ρ_0, ρ_1) provide a realistic range of noise levels, which will be used in the next stage to design noise experiments.

6.2 Impact on the Prediction Models

6.2.1 Overall results

Table 6.5 gives the mean MCC of the two batch models for all label sources, both evaluation settings and both ways of scoring. Table 6.6 gives the corresponding results for ORB. Each value is the mean over the 21 projects and ten seeds. The G-mean values are given in Table 6.7.

Some general observations can be made before the individual effects are examined.

The values are low in absolute terms. When the models are trained on the reference labels and scored against them, the MCC is between 0.10 and 0.24 for the batch models and below 0.10 for ORB. Predicting defect-introducing commits from the fourteen change metrics is evidently a difficult task on these projects.

The highest value in Table 6.5 is 0.413. It is obtained by JITLine trained on B-SZZ labels, evaluated with random cross-validation and scored against its own training labels. This is the configuration that corresponds most closely to common practice in the litera-

Table 6.5: Mean MCC of the batch models. “Ref.” denotes scoring against the reference labels and “Self” denotes scoring against the training labels. For models trained on the reference labels both are the same.

Model	Training labels	Random k -fold		Chronological split	
		Ref.	Self	Ref.	Self
JITLine	Reference	0.243	–	0.102	–
	B-SZZ	0.175	0.413	0.132	0.223
	AG-SZZ	0.116	0.304	0.090	0.135
	MA-SZZ	0.127	0.330	0.079	0.140
	L-SZZ	0.139	0.243	0.080	0.148
	R-SZZ	0.113	0.180	0.063	0.092
	RA-SZZ	0.116	0.307	0.086	0.140
LApredict	Reference	0.206	–	0.173	–
	B-SZZ	0.194	0.351	0.163	0.292
	AG-SZZ	0.200	0.247	0.168	0.197
	MA-SZZ	0.199	0.263	0.166	0.200
	L-SZZ	0.208	0.268	0.171	0.217
	R-SZZ	0.202	0.178	0.173	0.167
	RA-SZZ	0.200	0.260	0.168	0.214

Table 6.6: Mean MCC of ORB. For the prequential setting the time-averaged and the terminal value are given, both scored against the reference labels.

Training labels	Chronological-online		Prequential with latency	
	Ref.	Self	Time-averaged	Terminal
Reference	0.078	–	0.097	0.069
B-SZZ	0.077	0.161	0.058	0.057
AG-SZZ	0.018	0.068	0.036	0.012
MA-SZZ	0.022	0.072	0.035	–0.003
L-SZZ	0.008	0.070	0.058	0.021
R-SZZ	0.011	0.035	0.032	0.011
RA-SZZ	0.019	0.067	0.039	0.019

Table 6.7: Mean G-mean of the three models, scored against the reference labels. For ORB in the prequential setting the time-averaged value is given.

Training labels	JITLine		LApredict		ORB	
	<i>k</i> -fold	Chrono.	<i>k</i> -fold	Chrono.	Chrono.-online	Prequential
Reference	0.698	0.519	0.677	0.639	0.346	0.525
B-SZZ	0.648	0.574	0.667	0.654	0.468	0.503
AG-SZZ	0.595	0.534	0.674	0.655	0.363	0.476
MA-SZZ	0.604	0.512	0.673	0.654	0.364	0.477
L-SZZ	0.603	0.517	0.672	0.629	0.278	0.493
R-SZZ	0.591	0.502	0.675	0.654	0.317	0.472
RA-SZZ	0.591	0.528	0.673	0.650	0.342	0.473

ture.

The ranking of the two batch models depends on the evaluation setting. With random cross-validation and reference labels, JITLine (0.243) is better than LApredict (0.206). With the chronological split it is the other way round: LApredict reaches 0.173 and JITLine only 0.102. The model with one feature is thus the best of the three models in the chronological setting. A paired comparison confirms that it is significantly better than ORB in the chronological-online setting (Hodges–Lehmann estimate $+0.094$, 95% CI $[+0.061, +0.133]$, corrected $p = 0.006$), while the difference between JITLine and ORB is not significant.

LApredict is hardly affected by the label source. Its reference-scored MCC in the chronological setting varies only between 0.163 and 0.173 over the seven label sources. JITLine and ORB vary much more.

The G-mean values in Table 6.7 lead to the same main conclusions as the MCC values. For the models trained on the reference labels, the G-mean of JITLine falls from 0.698 under random cross-validation to 0.519 under the chronological split, whereas that of LApredict falls only from 0.677 to 0.639. LApredict again has the highest values in the chronological setting and changes little with the label source (0.629 to 0.655). In the prequential setting, ORB trained on the reference labels has the highest G-mean (0.525), and its G-mean is higher than that of each SZZ-trained model in 16 or 17 of the 21 projects.

The two measures differ in one case. In the chronological-online setting, ORB trained on B-SZZ labels has a clearly higher G-mean (0.468) than ORB trained on the reference labels (0.346), although their MCC values are almost equal (0.077 and 0.078). The G-mean becomes zero when one of the two classes is not recognised at all. This happens in 21.4% of the runs of the reference-trained model in this setting, but in only 3.8% of the runs of the model trained on B-SZZ labels, which lowers the mean of the former. The

G-mean also does not take the precision of the predictions into account. For these reasons MCC is used for the statistical comparisons in the following sections, and the G-mean values are reported for completeness without statistical tests.

6.2.2 Effect of the evaluation setting

The first effect to be examined is the difference between random k -fold cross-validation and the chronological split. Table 6.8 gives the results of the paired comparison for both batch models and all label sources, scored against the reference labels.

Table 6.8: Random k -fold cross-validation compared with the chronological split (MCC scored against the reference labels). HL is the Hodges–Lehmann estimate of the difference, r the rank-biserial correlation, and “Projects” the number of projects in which the k -fold value is higher. The last two columns give the Holm-corrected p -values within the family and over all tests.

Model	Labels	k -fold	Chrono.	HL	95% CI	r	Projects	p (family)	p (global)
JITLine	Reference	0.243	0.102	+0.137	[+0.105, +0.172]	+1.000	21/21	<0.001	<0.001
	B-SZZ	0.175	0.132	+0.055	[+0.019, +0.087]	+0.558	18/21	0.215	0.835
	AG-SZZ	0.116	0.090	+0.035	[−0.009, +0.064]	+0.377	15/21	0.402	1.000
	MA-SZZ	0.127	0.079	+0.050	[+0.013, +0.093]	+0.576	17/21	0.214	0.721
	L-SZZ	0.139	0.080	+0.067	[+0.022, +0.090]	+0.792	18/21	0.009	0.039
	R-SZZ	0.113	0.063	+0.051	[+0.026, +0.076]	+0.706	18/21	0.039	0.161
	RA-SZZ	0.116	0.086	+0.042	[−0.005, +0.078]	+0.420	15/21	0.402	1.000
LApredict	Reference	0.206	0.173	+0.034	[+0.001, +0.065]	+0.489	13/21	0.402	1.000
	B-SZZ	0.194	0.163	+0.036	[−0.001, +0.067]	+0.463	13/21	0.402	1.000
	AG-SZZ	0.200	0.168	+0.036	[−0.002, +0.069]	+0.429	14/21	0.402	1.000
	MA-SZZ	0.199	0.166	+0.037	[+0.001, +0.069]	+0.489	15/21	0.402	1.000
	L-SZZ	0.208	0.171	+0.041	[+0.010, +0.068]	+0.576	15/21	0.214	0.721
	R-SZZ	0.202	0.173	+0.036	[−0.004, +0.064]	+0.403	15/21	0.402	1.000
	RA-SZZ	0.200	0.168	+0.036	[+0.000, +0.070]	+0.463	15/21	0.402	1.000

In all fourteen comparisons the MCC obtained with random cross-validation is higher than the MCC obtained with the chronological split. The size of the difference depends strongly on the model.

For JITLine trained on the reference labels the difference is 0.137 (95% CI [+0.105, +0.172]). The rank-biserial correlation is +1.000, which means that the cross-validation result is higher in every one of the 21 projects. This comparison remains significant after the correction over all 74 tests. The MCC measured with cross-validation (0.243) is more than twice the MCC measured with the chronological split (0.102). For JITLine trained on the SZZ labels the difference is smaller (0.035 to 0.067) and is significant after the global correction only for L-SZZ.

For LApredict the difference is between 0.034 and 0.041 for all label sources, and none of the comparisons is significant after correction.

Figure 6.5 shows the values for the individual projects for the models trained on the

reference labels. For JITLine every line goes down. For LAPredict the lines are much flatter, and in several projects the chronological value is even higher.

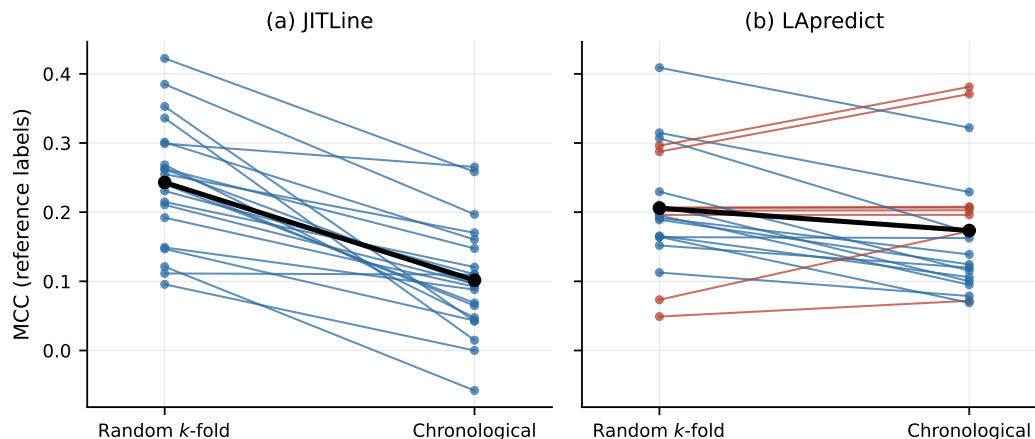


Figure 6.5: MCC of each project under random k -fold cross-validation and under the chronological split, for models trained on the reference labels. The thick line shows the mean.

The explanation is that in random cross-validation the model is trained on commits that lie close in time to the test commits, both before and after them. Commits that are close in time tend to be similar, because they concern the same parts of the code and are made by the same developers. A random forest can make use of this similarity. A logistic regression with a single feature has almost no capacity to do so. The overestimation caused by random cross-validation therefore increases with the capacity of the model. A consequence is that a comparison of a simple and a complex model under random cross-validation favours the complex model. In the present data the order of the two models is reversed when the evaluation respects time.

6.2.3 Effect of scoring against the training labels

The second effect is the difference between scoring a model against its own training labels and scoring it against the reference labels. Table 6.9 gives the comparison for the two batch models in both evaluation settings, and Figure 6.6 shows the mean differences.

The largest difference in the whole study is found for JITLine trained on B-SZZ labels under random cross-validation. The self-scored MCC is 0.413 and the reference-scored MCC is 0.175. The Hodges–Lehmann estimate of the difference is 0.230 (95% CI $[+0.189, +0.278]$), and the self-scored value is higher in 20 of the 21 projects. A reader who sees only the self-scored value would conclude that the model is more than twice as good as it is with respect to the reference.

Under random cross-validation the difference is significant for JITLine with five of the six variants after the global correction, and for AG-SZZ, MA-SZZ and RA-SZZ it is positive in all 21 projects. For LAPredict it is significant only with B-SZZ. With R-SZZ

Table 6.9: Self-scored MCC compared with reference-scored MCC for the batch models. “Projects” is the number of projects in which the self-scored value is higher. The other columns are as in Table 6.8.

Model	Labels	Self	Ref.	HL	95% CI	r	Projects	p (family)	p (global)
<i>Random k-fold cross-validation</i>									
JITLine	B-SZZ	0.413	0.175	+0.230	[+0.189, +0.278]	+0.991	20/21	<0.001	<0.001
	AG-SZZ	0.304	0.116	+0.180	[+0.120, +0.246]	+1.000	21/21	<0.001	<0.001
	MA-SZZ	0.330	0.127	+0.198	[+0.139, +0.265]	+1.000	21/21	<0.001	<0.001
	L-SZZ	0.243	0.139	+0.108	[+0.070, +0.139]	+0.939	18/21	<0.001	0.001
	R-SZZ	0.180	0.113	+0.065	[+0.030, +0.100]	+0.758	17/21	0.037	0.073
	RA-SZZ	0.307	0.116	+0.186	[+0.122, +0.252]	+1.000	21/21	<0.001	<0.001
LApredict	B-SZZ	0.351	0.194	+0.155	[+0.121, +0.203]	+0.957	20/21	<0.001	<0.001
	AG-SZZ	0.247	0.200	+0.051	[−0.012, +0.110]	+0.351	12/21	0.943	1.000
	MA-SZZ	0.263	0.199	+0.069	[+0.006, +0.119]	+0.498	13/21	0.644	1.000
	L-SZZ	0.268	0.208	+0.062	[+0.008, +0.116]	+0.541	14/21	0.522	0.899
	R-SZZ	0.178	0.202	−0.024	[−0.066, +0.026]	−0.221	9/21	0.971	1.000
	RA-SZZ	0.260	0.200	+0.060	[+0.007, +0.117]	+0.481	15/21	0.711	1.000
<i>Chronological split</i>									
JITLine	B-SZZ	0.223	0.132	+0.098	[+0.051, +0.141]	+0.758	17/21	0.037	0.073
	AG-SZZ	0.135	0.090	+0.041	[−0.004, +0.088]	+0.420	13/21	0.766	1.000
	MA-SZZ	0.140	0.079	+0.054	[+0.011, +0.097]	+0.628	15/21	0.233	0.415
	L-SZZ	0.148	0.080	+0.086	[+0.045, +0.113]	+0.628	18/21	0.233	0.415
	R-SZZ	0.092	0.063	+0.034	[−0.005, +0.073]	+0.403	15/21	0.777	1.000
	RA-SZZ	0.140	0.086	+0.052	[+0.007, +0.100]	+0.524	14/21	0.561	1.000
LApredict	B-SZZ	0.292	0.163	+0.137	[+0.104, +0.181]	+0.905	20/21	0.002	0.003
	AG-SZZ	0.197	0.168	+0.036	[−0.032, +0.101]	+0.255	14/21	0.971	1.000
	MA-SZZ	0.200	0.166	+0.035	[−0.012, +0.098]	+0.299	13/21	0.971	1.000
	L-SZZ	0.217	0.171	+0.064	[−0.016, +0.110]	+0.446	16/21	0.760	1.000
	R-SZZ	0.167	0.173	+0.006	[−0.056, +0.040]	+0.082	11/21	0.971	1.000
	RA-SZZ	0.214	0.168	+0.065	[−0.007, +0.117]	+0.446	17/21	0.760	1.000

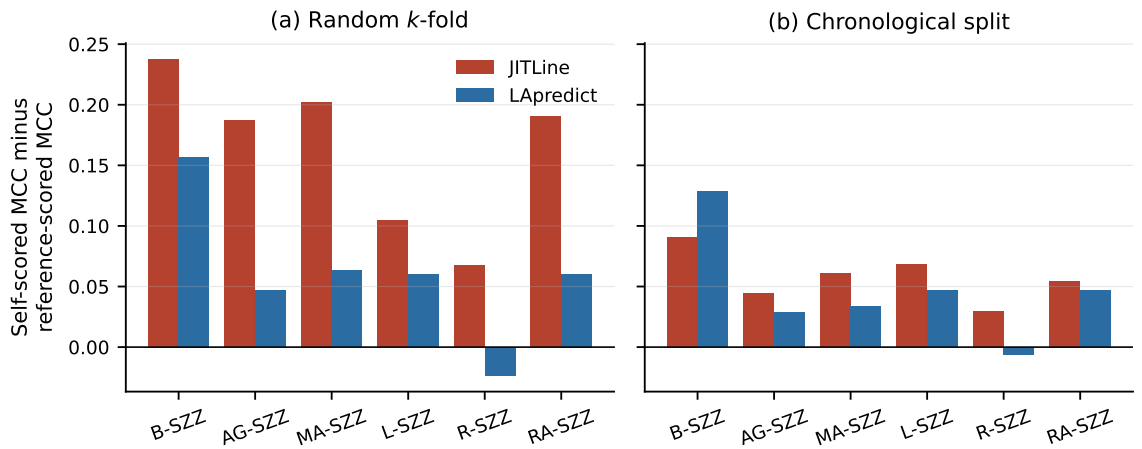


Figure 6.6: Difference between the self-scored and the reference-scored MCC for each SZZ variant and both batch models.

the difference for LAPredict is even slightly negative.

Under the chronological split the differences are smaller. For JITLine with B-SZZ the estimate is 0.098, which is significant within its family but not after the global correction ($p = 0.073$). For LAPredict with B-SZZ it is 0.137 and significant. For ORB, the difference in the chronological-online setting with B-SZZ is 0.082 (95% CI $[+0.054, +0.115]$, corrected $p = 0.004$), and in the prequential setting with the time-averaged summary it is 0.053 for B-SZZ, 0.042 for MA-SZZ and 0.036 for RA-SZZ, all of which are significant after the global correction.

Two patterns can be seen in these results. The difference is largest for B-SZZ, the variant with the highest false-alarm rate, and smallest for R-SZZ and L-SZZ, which have low false-alarm rates. And for the refined variants it is much larger for the random forest than for the logistic regression. Both patterns are consistent with the following explanation. The false positives of SZZ are not distributed at random over the commits. They are commits whose lines were later touched by a bug-fixing commit, which is more likely for large changes and for frequently modified files. A model can learn to recognise such commits. When it is scored against the SZZ labels, it is rewarded for this, although these commits are clean according to the reference. A model with high capacity can learn these regularities better than a model with a single feature. The case of B-SZZ with LAPredict fits this as well, since the single feature of LAPredict is the size of the change.

This explanation was tested directly with the analysis described in Section 5.10. A random forest was trained to distinguish the false positives of B-SZZ from its true positives, using only the commits flagged by B-SZZ. The mean AUC over the projects is 0.599. With randomly permuted training labels it is 0.514. The difference is significant (Hodges–Lehmann estimate $+0.088$, 95% CI $[+0.041, +0.131]$, higher in 17 of 21 projects, $p = 0.002$). The false positives of B-SZZ can therefore be distinguished from its true positives better than by chance, which confirms that the errors of SZZ have a structure that a model can learn. The AUC of 0.60 is modest, however, so this structure is not strong, and it probably does not account for the whole difference between the two ways of scoring.

6.2.4 Effect of the label source in the online setting

The third effect is the one for which the study was designed: the loss in performance when a model is trained on SZZ labels instead of the reference labels, measured in the most realistic setting. Table 6.10 compares ORB trained on the reference labels with ORB trained on each SZZ variant in the prequential setting with verification latency. All models are scored against the reference labels.

With the time-averaged MCC, which is the primary summary, the model trained on the reference labels reaches 0.097 and is better than the models trained on all six variants.

Table 6.10: ORB trained on the reference labels compared with ORB trained on each SZZ variant, in the prequential setting with verification latency. HL is the estimate of the difference (reference minus variant). “Projects” is the number of projects in which the reference-trained model is better.

Training labels	MCC	HL	95% CI	r	Projects	p (family)	p (global)
<i>Time-averaged MCC (reference-trained model: 0.097)</i>							
B-SZZ	0.058	+0.041	[+0.022, +0.052]	+0.879	18/21	<0.001	0.006
AG-SZZ	0.036	+0.060	[+0.029, +0.085]	+0.818	17/21	<0.001	0.024
MA-SZZ	0.035	+0.064	[+0.039, +0.085]	+0.896	18/21	<0.001	0.004
L-SZZ	0.058	+0.034	[+0.014, +0.055]	+0.723	16/21	0.002	0.124
R-SZZ	0.032	+0.060	[+0.042, +0.082]	+0.948	19/21	<0.001	<0.001
RA-SZZ	0.039	+0.058	[+0.036, +0.081]	+0.879	18/21	<0.001	0.006
<i>Terminal MCC (reference-trained model: 0.069)</i>							
B-SZZ	0.057	+0.015	[−0.023, +0.052]	+0.177	13/21	0.495	1.000
AG-SZZ	0.012	+0.060	[+0.020, +0.101]	+0.636	15/21	0.045	0.397
MA-SZZ	−0.003	+0.072	[+0.019, +0.128]	+0.636	16/21	0.045	0.397
L-SZZ	0.021	+0.044	[+0.011, +0.094]	+0.602	15/21	0.045	0.538
R-SZZ	0.011	+0.049	[+0.014, +0.105]	+0.654	16/21	0.043	0.327
RA-SZZ	0.019	+0.045	[+0.015, +0.093]	+0.619	17/21	0.045	0.443

The estimated loss is between 0.034 (L-SZZ) and 0.064 (MA-SZZ). All six confidence intervals exclude zero, the reference-trained model is better in 16 to 19 of the 21 projects, and five of the six comparisons remain significant after the correction over all tests. The exception is L-SZZ, which is significant within its family only. In relative terms, the SZZ-trained models retain between about one third and three fifths of the MCC of the reference-trained model. Figure 6.7 shows the means and the estimated differences.

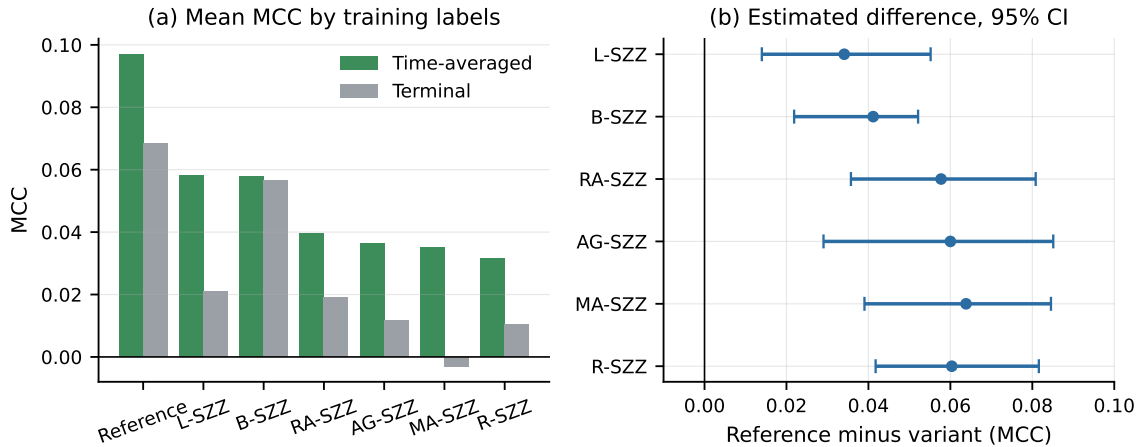


Figure 6.7: ORB in the prequential setting with verification latency. (a) Mean MCC for each source of training labels with both summaries. (b) Hodges–Lehmann estimate and 95% confidence interval of the difference between the reference-trained model and each SZZ-trained model (time-averaged MCC).

The comparison with B-SZZ deserves particular attention. As explained in Section 4.4, B-SZZ and the reference labels use the same blame mechanism and differ in the

lines from which the tracing starts. The loss of 0.041 MCC (95% CI $[+0.022, +0.052]$) can therefore be read as an estimate of what the knowledge of the actual bug-fixing lines is worth to an online learner in this setting.

The size of the loss is not related in a simple way to the recall of the variants. L-SZZ, with a recall of 0.267, and B-SZZ, with a recall of 0.641, show almost the same loss, while AG-SZZ, MA-SZZ and RA-SZZ, with recall values in between, show larger losses. The correlation between the mean loss and the recall over the six variants is -0.18 . Which property of the labels determines the loss cannot be decided from this comparison, because the variants differ from each other in several respects at the same time. This question is taken up in the next stage of the thesis.

Results with the terminal MCC. With the terminal MCC the reference-trained model is also better than all variants on average, and five of the six comparisons are significant within the family, although none remains significant after the global correction. The comparison with B-SZZ is not significant with this summary ($+0.015$, 95% CI $[-0.023, +0.052]$). This is the only comparison in the study for which the two summaries lead to different conclusions. Figure 6.8 shows why the terminal value is less suitable for such comparisons. Its spread over the projects is almost twice as large as that of the time-averaged value, because it is computed from the last hundred commits of each project only. The negative value for MA-SZZ in Table 6.6 is also a consequence of this and does not appear with the time-averaged summary.

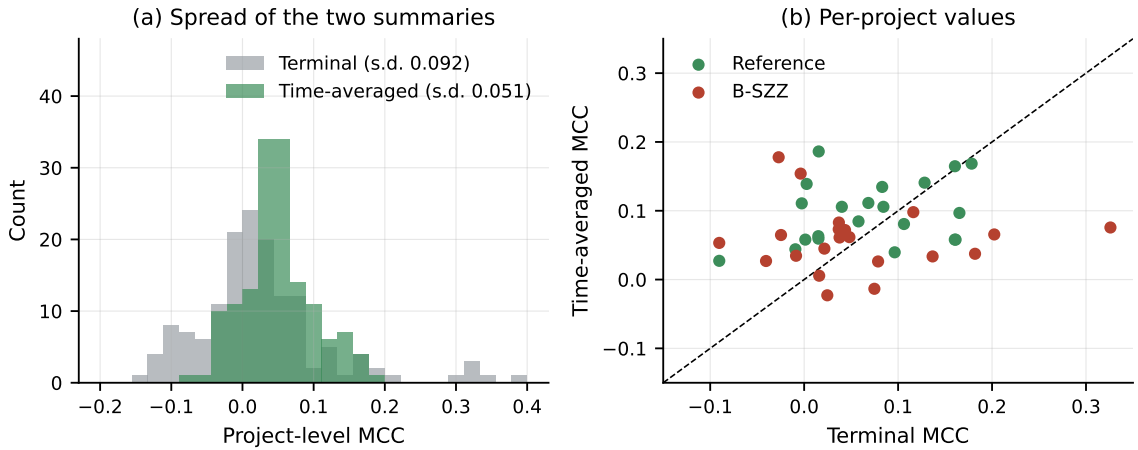


Figure 6.8: Comparison of the two summaries of the prequential MCC. (a) Distribution of the project-level values over all label sources. (b) Terminal and time-averaged value of each project for models trained on the reference labels and on B-SZZ labels.

Check on the availability of arrival times. For the reference labels an arrival time is available for only 67.8% of the defect-introducing commits, whereas all commits flagged by B-SZZ have one (Table 4.4). To check whether this difference affects the compari-

son, the missing arrival times were imputed from the observed latency distribution. With imputed arrival times the time-averaged MCC of the reference-trained model is 0.084 instead of 0.097. It is still higher than that of the B-SZZ-trained model (0.058). The mean difference is 0.026, the reference-trained model is better in 16 of 21 projects, and the difference is significant ($p = 0.010$). With the terminal MCC the difference is 0.004 and not significant, in line with the result without imputation. The advantage of the reference labels is thus reduced but not removed when the availability of arrival times is equalised. The reduction is expected: more than half of the imputed labels arrive after the waiting period, so the corresponding commits are first presented to the learner as clean.

6.2.5 Combined view of the three effects

Figure 6.9 puts the three effects together. It shows the MCC of six configurations, starting from the one that is closest to common practice and ending with the most realistic one.

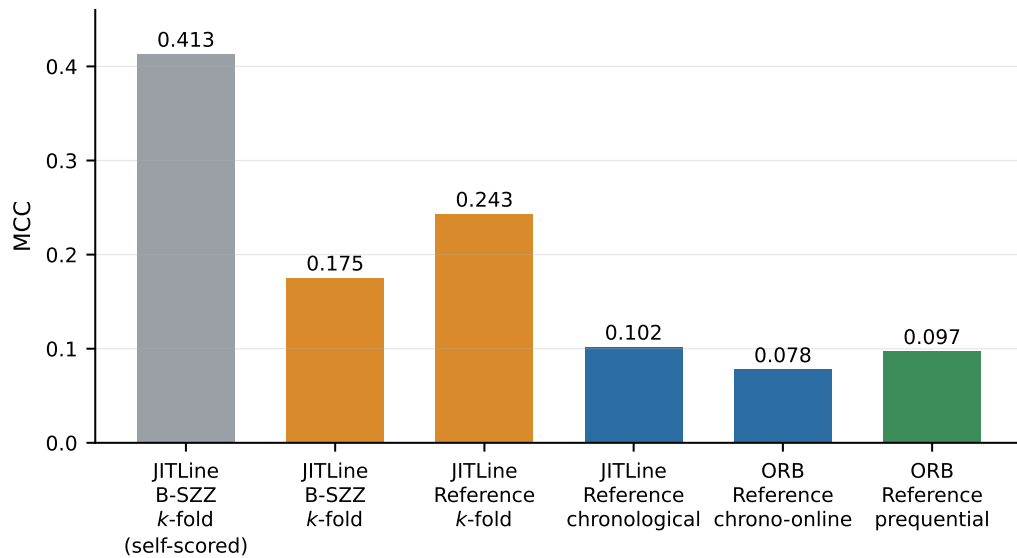


Figure 6.9: MCC under increasingly realistic conditions. Each bar is labelled with the model, the training labels and the evaluation setting. All bars except the first are scored against the reference labels.

JITLine trained on B-SZZ labels, evaluated with random cross-validation and scored against its own labels has an MCC of 0.413. When the same predictions are scored against the reference labels, the value is 0.175. When the model is trained on the reference labels and evaluated with the chronological split, it is 0.102. ORB trained on the reference labels reaches 0.078 in the chronological-online setting and 0.097 in the prequential setting with verification latency.

The value that would be reported under common practice is thus about four times the value obtained under realistic conditions with the best available labels. Of the three effects, the scoring against the training labels (0.230) and the random cross-validation

(0.137) are several times larger than the loss caused by training on SZZ labels in the online setting (0.034 to 0.064).

The figure should be read as a description and not as a decomposition. The bars differ in more than one respect. In particular, the step from the batch model to the online model changes the learner, the delivery of the labels and the part of the stream on which the model is tested. This step is examined next.

6.2.6 Label delay and model adaptation

In Table 6.6 ORB with reference labels reaches 0.078 in the chronological-online setting and 0.069 (terminal) or 0.097 (time-averaged) in the prequential setting. It would be tempting to conclude from these numbers that verification latency has little effect. Such a conclusion is not justified, because the two settings differ in three respects: in the prequential setting the labels are delayed, the model keeps learning, and the performance is measured over the whole stream with fading instead of over the second half.

To separate these factors, the experiment with four combinations described in Section 5.10 was carried out. Table 6.11 gives the mean MCC of the four cells and the paired comparisons, and Figure 6.10 shows the cell means.

Table 6.11: ORB with reference labels in the four combinations of learner (frozen or adaptive) and label delivery (immediate or delayed). MCC on the second half of the stream.

	Labels immediate Labels delayed			
Frozen learner	0.083	0.078		
Adaptive learner	0.127	0.101		

Comparison	HL	95% CI	<i>r</i>	<i>p</i>
Adaptive minus frozen, labels immediate	+0.042	[−0.002, +0.083]	+0.463	0.065
Adaptive minus frozen, labels delayed	+0.015	[−0.011, +0.044]	+0.221	0.393
Immediate minus delayed, adaptive learner	+0.023	[−0.011, +0.052]	+0.273	0.288
Immediate minus delayed, frozen learner	−0.012	[−0.065, +0.047]	−0.117	0.658
Interaction	−0.030	[−0.073, +0.024]	−0.247	0.338

The directions of the effects are as expected. An adaptive learner is better than a frozen one, and delayed labels reduce the performance of the adaptive learner (0.127 against 0.101). For the frozen learner the delay makes almost no difference. However, none of the comparisons is statistically significant. All confidence intervals include zero, including that of the interaction. With 21 projects, differences of this size, around 0.02 to 0.04 MCC, cannot be established reliably.

The result is therefore reported as inconclusive. It does not show that verification latency has no effect. It shows that the effect, if it exists, is too small to be detected with

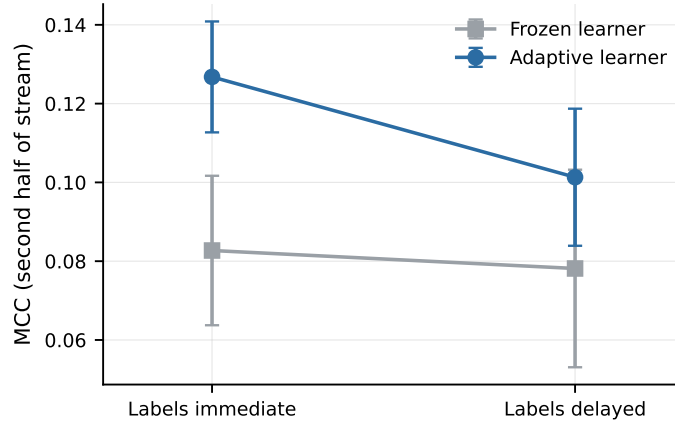


Figure 6.10: Mean MCC of the four combinations of learner and label delivery. The bars show one standard error over the projects.

this design. The experiment also shows that a design in which several things change at the same time, as in the comparison of two label sources or two settings, is not suitable for isolating the effect of latency. In the next stage, experiments are planned in which the learner and the stream are kept fixed and only the labels are changed in a controlled way.

6.2.7 A case in which B-SZZ labels give a better batch model

One result in Table 6.5 is contrary to expectation. In the chronological setting, JITLine trained on B-SZZ labels has a higher reference-scored MCC (0.132) than JITLine trained on the reference labels (0.102). Training on the noisier labels gives the better model. Figure 6.11 shows the difference for each project.

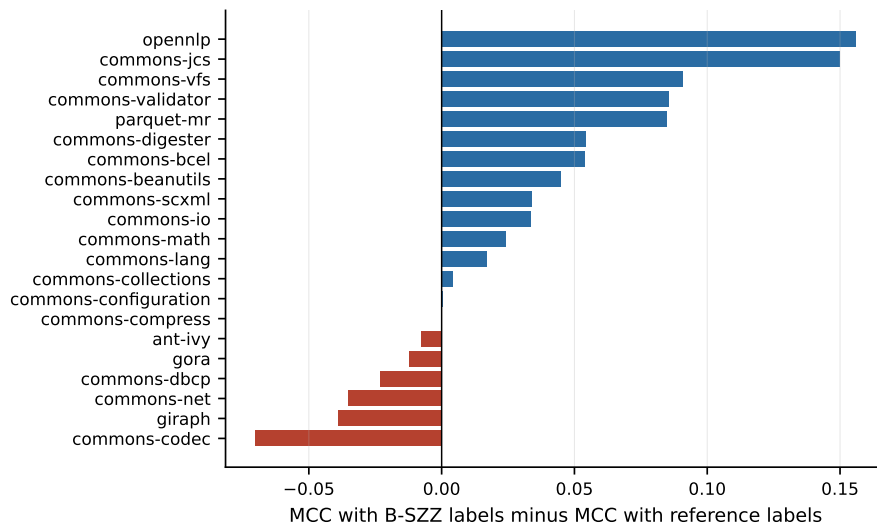


Figure 6.11: Difference between the MCC of JITLine trained on B-SZZ labels and JITLine trained on the reference labels, for each project (chronological split, scored against the reference labels).

The model trained on B-SZZ labels is better in 15 of the 21 projects. The differences are stable over the seeds. In 15 projects the mean difference is more than two standard

errors away from zero, and about 95% of the variance of the differences between projects is not due to the seeds. The effect was also present, with a similar size, when other procedures for selecting the decision threshold of the random forest were tried. It is therefore not a random fluctuation and not a consequence of the threshold selection.

An obvious hypothesis is that B-SZZ helps because it provides more positive training examples. The reference labels contain few positive examples in some projects, and a random forest may learn more from a larger, although noisier, positive class. To examine this, the difference was correlated with several characteristics of the projects (Table 6.12).

Table 6.12: Spearman correlation between project characteristics and the difference shown in Figure 6.11.

Project characteristic	Spearman ρ	p
Defect-introducing commits in the training half	−0.249	0.28
Defect-introducing commits in total	−0.157	0.50
Number of commits	−0.130	0.57
Share of reference labels with an arrival time	−0.086	0.71
Precision of B-SZZ	−0.082	0.72
Recall of B-SZZ	−0.013	0.96
Percentage of commits flagged by B-SZZ	+0.021	0.93
Reference defect rate	−0.005	0.98
B-SZZ flag rate divided by reference defect rate	+0.042	0.86

None of the characteristics is significantly correlated with the difference. The correlation with the number of defect-introducing commits in the training half has the sign that the hypothesis predicts but is weak. There is also a project that contradicts the hypothesis. In `opennlp` the difference is the largest of all projects (+0.156), but B-SZZ flags fewer commits there (6.9%) than the reference labels (8.4%), so it cannot help by providing more positive examples.

The effect is limited to the batch model. For ORB in the prequential setting the reference-trained model is better than the B-SZZ-trained model in 18 of 21 projects (Table 6.10). The observation is reported here because it is stable and may be of interest for further study. Its cause has not been established, and no explanation is claimed.

6.2.8 Robustness checks

Fading factor and warm-up period. The comparison of the label sources in the prequential setting was repeated for five fading factors and four lengths of an excluded warm-up period, giving 20 settings for each of the six variants. Figure 6.12 shows the estimated differences. In all 120 comparisons the reference-trained model is better, the confidence interval excludes zero, and the comparison remains significant after a Holm correction

over the 120 comparisons. The estimated loss for B-SZZ lies between 0.027 and 0.044 over the 20 settings, and the MCC of the reference-trained model lies between 0.090 and 0.114. The conclusion of Section 6.2.4 does therefore not depend on the choice of the fading factor or on whether the beginning of the stream is included. These 120 comparisons are a check of one result under different settings and should not be counted as separate findings.

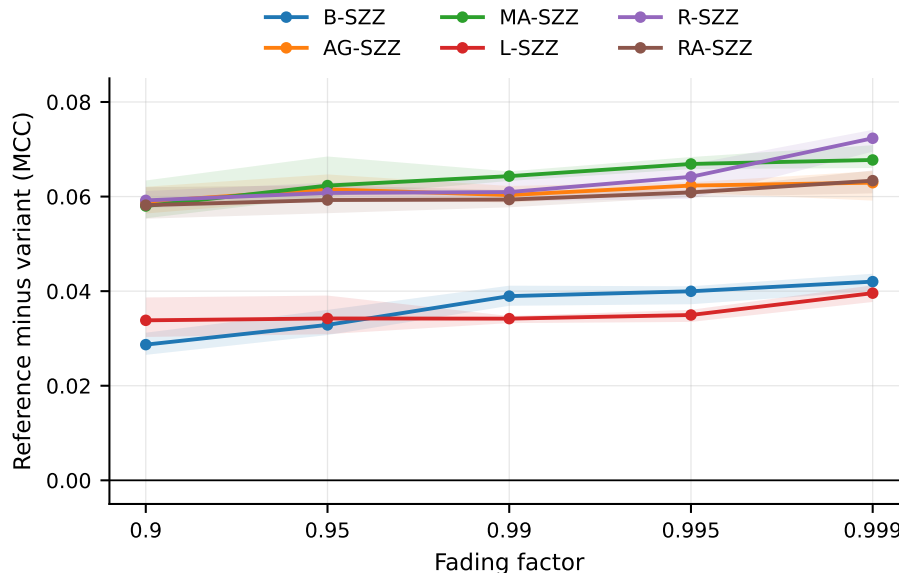


Figure 6.12: Estimated difference between the reference-trained and the SZZ-trained model for different fading factors. The lines show the mean over the four warm-up settings and the shaded bands show their range.

Exclusion of small projects. All means in this chapter give the same weight to every project, although the projects differ greatly in size. Eight projects have fewer than 25 defect-introducing commits in the second half of their history. Table 6.13 repeats the main comparisons without these projects. The estimates of the three main effects change by at most 0.002, and all three remain significant. One result is weaker without the small projects: the difference between self-scored and reference-scored MCC for JITLine with B-SZZ in the chronological setting decreases from 0.098 to 0.075 and is no longer significant.

6.2.9 Discussion and answer to RQ2

RQ2 asked how the choice of the label source affects the measured performance of JIT-SDP models under different evaluation settings.

The label source has a clear effect when the models are evaluated realistically. An online learner trained on the labels of any of the six SZZ variants performs worse, with respect to the reference labels, than the same learner trained on the reference labels. The

Table 6.13: Main comparisons for all projects and for the 13 projects with at least 25 defect-introducing commits in the second half of their history.

Comparison	All 21 projects	13 projects
Random k -fold against chronological split (JITLine, reference labels)	+0.137 [+0.105, +0.172]	+0.136 [+0.095, +0.171]
Self-scored against reference-scored (JITLine, B-SZZ, k -fold)	+0.230 [+0.189, +0.278]	+0.232 [+0.156, +0.322]
Self-scored against reference-scored (JITLine, B-SZZ, chronological)	+0.098 [+0.051, +0.141]	+0.075 [+0.002, +0.135]
Reference against B-SZZ training labels (ORB, prequential)	+0.041 [+0.022, +0.052]	+0.043 [+0.021, +0.067]

loss is between 0.034 and 0.064 MCC. Since the reference-trained model itself reaches only 0.097, this is a substantial part of the performance that can be achieved.

The evaluation procedure has a larger effect on the measured performance than the label source. Random cross-validation increases the MCC of the random forest model by 0.137, and scoring against the training labels increases it by up to 0.230. Both effects are larger for the model with higher capacity. Under these two conditions the differences between label sources and between models can be hidden or even reversed. For example, under random cross-validation JITLine appears better than LAPredict, and under the chronological split LAPredict is clearly better.

The two issues are connected. Scoring against the training labels is done in practice because no other labels are available, and this is so because the labels are produced by a heuristic. The weakness of the labels thus leads to a weakness of the evaluation. A reference label set that is kept apart from training, as used in this work, makes the size of this effect visible.

Some limits of these conclusions should be stated. The absolute performance of all models is low, so the results say little about how useful JIT-SDP would be with better labels. The results do not show which property of the SZZ labels causes the loss in the online setting. And the experiment on label delay and model adaptation remained inconclusive. These points define the work of the next stage.

6.3 Summary of Findings

Table 6.14 summarises the main findings of the first stage together with the evidence for each.

Table 6.14: Summary of the main findings.

Finding	Evidence
1 The labels of all six SZZ variants differ strongly from the reference labels.	Precision 0.183 to 0.272; recall 0.267 to 0.641; κ with the reference 0.158 to 0.202
2 The noise is class-conditional and depends on the variant.	ρ_0 from 0.067 to 0.263; ρ_1 from 0.359 to 0.733
3 The variants agree with each other far more than with the reference.	κ between variants 0.33 to 0.93
4 A large part of the false negatives of the refined variants is caused by their own filtering.	30% to 60% of the true positives of B-SZZ are removed
5 Random cross-validation overestimates the performance, more so for the complex model.	JITLine +0.137 MCC in 21 of 21 projects; LAPredict +0.034, not significant
6 Scoring against the training labels overestimates the performance further.	Up to +0.230 MCC for JITLine with B-SZZ; the errors of B-SZZ are learnable (AUC 0.60 against 0.51)
7 In the online setting with verification latency, training on SZZ labels reduces the performance.	Loss of 0.034 to 0.064 MCC; significant for five of six variants after global correction
8 The effect of label delay could not be separated reliably with 21 projects.	All comparisons of the four-cell experiment not significant

Chapter 7

Conclusion and Future Work

This chapter concludes the report. It summarises the work of the first stage and its conclusions, discusses the threats to the validity of the results and describes the work planned for the next stage of the thesis.

7.1 Conclusions

The thesis deals with the effect of the label noise introduced by the SZZ algorithm on just-in-time software defect prediction. In the first stage, two studies were carried out on the JIT-Defects4J dataset, which contains 27,319 commits from 21 Apache Java projects and provides reference labels derived from manually validated bug-fixing lines.

In the first study, the labels of six SZZ variants were generated for all projects and compared with the reference labels. The comparison showed that the labels of all variants differ considerably from the reference labels. The precision of the variants is between 18.3% and 27.2%, and their recall is between 26.7% and 64.1%. The errors are of two types whose proportions depend on the variant. The original algorithm, B-SZZ, flags many clean commits, whereas the variants that keep only one candidate commit, L-SZZ and R-SZZ, miss about 70% of the defect-introducing commits. The refined variants reduce the false positives of B-SZZ, but a considerable part of their false negatives is caused by their own filtering rules. The variants agree with each other much more than with the reference labels, which shows that agreement between SZZ variants is not a sign of correct labels.

In the second study, three prediction models were trained on the reference labels and on the labels of each variant and were evaluated in four settings. The study led to three main results. First, random k -fold cross-validation overestimates the performance compared with a chronological split. For the random forest model the overestimation is 0.137 MCC and occurs in all 21 projects, while for the logistic regression model with a single feature it is small. Second, scoring a model against its own SZZ training labels over-

estimates the performance further, by up to 0.230 MCC. An additional analysis showed that the errors of B-SZZ can be predicted from the change metrics better than by chance, which explains why a model with high capacity benefits from this way of scoring. Third, when an online learner is evaluated on the commit stream with verification latency and is scored against the reference labels, training on the labels of any SZZ variant gives a lower performance than training on the reference labels. The loss is between 0.034 and 0.064 MCC, compared with an MCC of 0.097 for the model trained on the reference labels.

Taken together, the results show that the performance values obtained with the common experimental procedure are much higher than the values obtained under realistic conditions. In the present data, the first is about four times the second. The larger part of this difference is due to the evaluation procedure and a smaller part to the label noise. The label noise nevertheless causes a loss that is large in relation to the performance that an online learner can actually achieve.

The research questions RQ1 and RQ2 have thus been answered, and objectives 1 to 4 of Chapter 3 have been achieved.

7.2 Threats to Validity

7.2.1 Construct validity

The most important threat concerns the reference labels. They were obtained by applying blame to manually validated bug-fixing lines and are not a manual judgement about each commit. Errors of the blame step are shared by the reference labels and the SZZ labels and are not detected. The measured noise is therefore a lower bound. In addition, the set of bug-fixing commits given to the SZZ variants was taken from the FIX metric of the dataset and may not be identical to the set from which the reference labels were derived. A part of the disagreement between SZZ and the reference, in particular a part of the false negatives of B-SZZ, may be due to this difference and not to the algorithm.

A second threat concerns the arrival times of the labels. The dataset does not provide the link between a defect-introducing commit and its fix, so the arrival times of the reference labels were taken from the SZZ mappings. For 32.2% of the reference defect-introducing commits no arrival time is available. The check with imputed arrival times (Section 6.2.4) shows that the incomplete coverage does not explain the advantage of the reference labels, but the arrival times that are available may themselves contain errors. All results of the prequential setting are affected by this.

A third point is the summary of the prequential MCC. Neither the terminal nor the time-averaged value is an established standard for MCC. Both were reported, and the conclusions were checked for different fading factors.

7.2.2 Internal validity

The models used are implementations made or configured for this work. The random forest model follows the design of JITLine but uses only the change metrics, and the ORB implementation uses logistic regression base learners. Other implementations might give different absolute values. The comparisons in this work are made between conditions that use the same implementation, so they are less affected by this than the absolute values.

The hyperparameters of the models were not tuned for each label source. This avoids giving an advantage to one of the label sources, but it also means that the results hold for the chosen settings.

To reduce the risk of errors in the data processing, the consistency of the labels used for training with the labels analysed in the first study is checked automatically before the experiments are run.

7.2.3 External validity

The study uses 21 projects that are all written in Java and belong to the Apache Software Foundation, sixteen of them to the Apache Commons family. The results may not hold for projects in other languages, for industrial projects or for much larger projects. The restriction to this dataset is due to the need for reference labels at the level of commits, which are not available for other JIT-SDP datasets. The dataset excludes changes that only delete code, so defects introduced in this way are not covered. Only the fourteen change metrics were used as features. Models that use the content of the changes may be affected by label noise differently.

7.2.4 Conclusion validity

The statistical unit is the project, and there are 21 projects. Differences below about 0.04 MCC cannot be detected reliably with this number, as the experiment on label delay showed. Non-significant results in this report should therefore not be read as evidence that there is no effect.

The analysis was exploratory. The comparisons and their grouping into families were decided while the results were examined. To account for this, the correction for multiple comparisons was applied over all 74 tests, and the main conclusions rest on the results that remain significant after this correction. All means are unweighted over the projects, which differ in size. The check without the small projects showed that the main results do not depend on them.

7.3 Future Work

The first stage has shown that SZZ labels reduce the performance of an online learner, but it has not shown why. The next stage of the thesis addresses RQ3 and RQ4.

7.3.1 Mechanism of the noise effect

The comparison of label sources cannot identify the cause of the loss, because two label sources differ in the number of false positives, in the number of false negatives and in the arrival times of the labels at the same time. Experiments are therefore planned in which only one property of the labels is changed at a time.

Two kinds of experiments are planned. In the first, label noise will be injected into the reference labels in a controlled way. The noise rates will be chosen according to the pairs (ρ_0, ρ_1) measured in this report, so that noise with mainly false positives, as in B-SZZ, and noise with mainly false negatives, as in L-SZZ, can be compared at different levels. In the second, the B-SZZ labels will be corrected with the help of the reference labels, once by removing only the false positives and once by restoring only the false negatives. The change in performance will show which type of error is responsible for the loss.

The results of the first stage indicate three precautions for these experiments. Since B-SZZ produces about eight false positives for every false negative, a comparison in which the same number of labels of each type is corrected should be included, so that the effect of the type of error is not confused with the effect of its amount. Since restored defect labels have no arrival time, a condition in which they are delivered without delay is needed. And to judge the role of verification latency, a condition in which all labels are delivered immediately should be included. In addition, the resampling rate of ORB will be recorded during the experiments, to examine whether the oversampling of wrongly labelled positive examples (Section 5.4) contributes to the loss.

7.3.2 Reducing the effect of the noise

Depending on the outcome of these experiments, a modification of the online learner will be designed that makes it less sensitive to the type of error that is found to be harmful. The modification must work without access to clean labels, since these are not available in practice. Two requirements will be set for it. It should improve the performance when the learner is trained on SZZ labels, and it should not reduce the performance when the labels are clean, because a user does not know in advance how noisy the labels are. The hypotheses, the comparisons and the criteria for accepting the method will be fixed before the experiments are run, so that the evaluation of the method is confirmatory and not exploratory.

7.3.3 Further directions

Some further extensions are possible if time permits. The analysis could be repeated with effort-aware performance measures, which take the cost of inspecting a commit into account. Other online learners could be included. The label quality analysis could be extended to other projects if suitable reference labels become available.

References

- [1] T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, “A systematic literature review on fault prediction performance in software engineering,” *IEEE Transactions on Software Engineering*, vol. 38, no. 6, pp. 1276–1304, 2012.
- [2] S. S. Rathore and S. Kumar, “A study on software fault prediction techniques,” *Artificial Intelligence Review*, vol. 51, no. 2, pp. 255–327, 2019.
- [3] Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, and N. Ubayashi, “A large-scale empirical study of just-in-time quality assurance,” *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 757–773, 2013.
- [4] A. Mockus and D. M. Weiss, “Predicting risk of software changes,” *Bell Labs Technical Journal*, vol. 5, no. 2, pp. 169–180, 2000.
- [5] S. Kim, E. J. Whitehead, Jr., and Y. Zhang, “Classifying software changes: Clean or buggy?” *IEEE Transactions on Software Engineering*, vol. 34, no. 2, pp. 181–196, 2008.
- [6] J. Śliwerski, T. Zimmermann, and A. Zeller, “When do changes induce fixes?” in *Proceedings of the International Workshop on Mining Software Repositories (MSR)*, 2005, pp. 1–5.
- [7] S. Herbold, A. Trautsch, B. Ledel, *et al.*, “A fine-grained data set and analysis of tangling in bug fixing commits,” *Empirical Software Engineering*, vol. 27, no. 6, article 125, 2022.
- [8] D. A. da Costa, S. McIntosh, W. Shang, U. Kulesza, R. Coelho, and A. E. Hassan, “A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes,” *IEEE Transactions on Software Engineering*, vol. 43, no. 7, pp. 641–657, 2017.
- [9] G. Rosa, L. Pascarella, S. Scalabrino, R. Tufano, G. Bavota, M. Lanza, and R. Oliveto, “Evaluating SZZ implementations through a developer-informed oracle,” in *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*, 2021.
- [10] D. Falessi, J. Huang, L. Narayana, J. F. Thai, and B. Turhan, “On the need of preserving order of data when validating within-project defect classifiers,” *Empirical Software Engineering*, vol. 25, pp. 4805–4830, 2020.
- [11] M. Tan, L. Tan, S. Dara, and C. Mayeux, “Online defect prediction for imbalanced data,” in *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, vol. 2, 2015, pp. 99–108.
- [12] G. G. Cabral, L. L. Minku, E. Shihab, and S. Mujahid, “Class imbalance evolution and verification latency in just-in-time software defect prediction,” in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 666–676.
- [13] Y. Zhao, K. Damevski, and H. Chen, “A systematic survey of just-in-time software defect prediction,” *ACM Computing Surveys*, vol. 55, no. 10, pp. 1–35, 2023.
- [14] S. McIntosh and Y. Kamei, “Are fix-inducing changes a moving target? A longitudinal case study of just-in-time defect prediction,” *IEEE Transactions on Software Engineering*, vol. 44, no. 5, pp. 412–428, 2018.
- [15] T. Hoang, H. K. Dam, Y. Kamei, D. Lo, and N. Ubayashi, “DeepJIT: An end-to-end deep learning framework for just-in-time defect prediction,” in *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*, 2019.
- [16] T. Hoang, H. J. Kang, D. Lo, and J. Lawall, “CC2Vec: Distributed representations of code

- changes,” in *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, 2020.
- [17] Z. Zeng, Y. Zhang, H. Zhang, and L. Zhang, “Deep just-in-time defect prediction: How far are we?” in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2021.
 - [18] C. Pornprasit and C. Tantithamthavorn, “JITLine: A simpler, better, faster, finer-grained just-in-time defect prediction,” in *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*, 2021, pp. 369–379.
 - [19] Y. Yang, Y. Zhou, J. Liu, Y. Zhao, H. Lu, L. Xu, B. Xu, and H. Leung, “Effort-aware just-in-time defect prediction: Simple unsupervised models could be better than supervised models,” in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2016, pp. 157–168.
 - [20] G. Rodríguez-Pérez, G. Robles, A. Serebrenik, A. Zaidman, D. M. Germán, and J. M. González-Barahona, “How bugs are born: A model to identify how bugs are introduced in software components,” *Empirical Software Engineering*, vol. 25, pp. 1294–1340, 2020.
 - [21] S. Kim, T. Zimmermann, K. Pan, and E. J. Whitehead, Jr., “Automatic identification of bug-introducing changes,” in *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2006, pp. 81–90.
 - [22] S. Davies, M. Roper, and M. Wood, “Comparing text-based and dependence-based approaches for determining the origins of bugs,” *Journal of Software: Evolution and Process*, vol. 26, no. 1, pp. 107–139, 2014.
 - [23] E. C. Neto, D. A. da Costa, and U. Kulesza, “The impact of refactoring changes on the SZZ algorithm: An empirical study,” in *Proceedings of the 25th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2018, pp. 380–390.
 - [24] G. Rodríguez-Pérez, G. Robles, and J. M. González-Barahona, “Reproducibility and credibility in empirical software engineering: A case study based on a systematic literature review of the use of the SZZ algorithm,” *Information and Software Technology*, vol. 99, pp. 164–176, 2018.
 - [25] G. Rosa, L. Pascarella, S. Scalabrino, R. Tufano, G. Bavota, M. Lanza, and R. Oliveto, “A comprehensive evaluation of SZZ variants through a developer-informed oracle,” *Journal of Systems and Software*, vol. 202, article 111729, 2023.
 - [26] K. Herzig and A. Zeller, “The impact of tangled code changes,” in *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*, 2013, pp. 121–130.
 - [27] S. Kim, H. Zhang, R. Wu, and L. Gong, “Dealing with noise in defect prediction,” in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, 2011, pp. 481–490.
 - [28] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, A. Ihara, and K. Matsumoto, “The impact of mislabelling on the performance and interpretation of defect prediction models,” in *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, 2015, pp. 812–823.
 - [29] Y. Fan, X. Xia, D. A. da Costa, D. Lo, A. E. Hassan, and S. Li, “The impact of changes mislabeled by SZZ on just-in-time defect prediction,” *IEEE Transactions on Software Engineering*, vol. 47, no. 8, pp. 1559–1586, 2021.
 - [30] J. Gama, R. Sebastião, and P. P. Rodrigues, “On evaluating stream learning algorithms,” *Machine Learning*, vol. 90, no. 3, pp. 317–346, 2013.
 - [31] G. G. Cabral and L. L. Minku, “Towards reliable online just-in-time software defect prediction,” *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1342–1358, 2023.
 - [32] S. Tabassum, L. L. Minku, D. Feng, G. G. Cabral, and L. Song, “An investigation of cross-project learning in online just-in-time software defect prediction,” in *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 554–565.

- [33] J. Yao and M. Shepperd, "Assessing software defection prediction performance: Why using the Matthews correlation coefficient matters," in *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2020, pp. 120–129.
- [34] B. W. Matthews, "Comparison of the predicted and observed secondary structure of T4 phage lysozyme," *Biochimica et Biophysica Acta (BBA) – Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [35] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, article 6, 2020.
- [36] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [37] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
- [38] N. C. Oza and S. Russell, "Online bagging and boosting," in *Proceedings of the 8th International Workshop on Artificial Intelligence and Statistics (AISTATS)*, 2001, pp. 229–236.
- [39] S. Wang, L. L. Minku, and X. Yao, "Resampling-based ensemble methods for online class imbalance learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 5, pp. 1356–1368, 2015.
- [40] N. Natarajan, I. S. Dhillon, P. Ravikumar, and A. Tewari, "Learning with noisy labels," in *Advances in Neural Information Processing Systems (NIPS)*, 2013.
- [41] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. Tsang, and M. Sugiyama, "Co-teaching: Robust training of deep neural networks with extremely noisy labels," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [42] C. G. Northcutt, L. Jiang, and I. L. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *Journal of Artificial Intelligence Research*, vol. 70, pp. 1373–1411, 2021.
- [43] H. Keshavarz and M. Nagappan, "ApacheJIT: A large dataset for just-in-time defect prediction," in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, 2022.
- [44] P. Mahbub, O. Shuvo, and M. M. Rahman, "Defectors: A large, diverse Python dataset for defect prediction," in *Proceedings of the 20th International Conference on Mining Software Repositories (MSR)*, 2023.
- [45] C. Ni, W. Wang, K. Yang, X. Xia, K. Liu, and D. Lo, "The best of both worlds: Integrating semantic features with expert features for defect prediction and localization," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022, pp. 672–683.
- [46] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [47] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [48] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [49] J. L. Hodges and E. L. Lehmann, "Estimates of location based on rank tests," *The Annals of Mathematical Statistics*, vol. 34, no. 2, pp. 598–611, 1963.
- [50] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [51] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.